PARKING SLOT RESERVATION & MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

ABINASH G

(2116240701007)

in partial fulfilment of the End Semester Examination for the course

CB23F35 – LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

In

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**PARKING SLOT RESERVATION & MANAGEMENT SYSTEM**" is the bona fide work of **ABINASH G (**240701007), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr.J.MANORANJANI
HEAD OF THE DEPARTMENT
Department of CSE
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr.J.JINU SOPHIA
COURSE FACULTY
Associate Professor
Department of CSE
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, **ABINASH G (240701007)**, hereby declare that the project report titled "**Parking Slot Reservation & Management System**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and approval process — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
ABINASH G
240701007

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Thiru. S. Meganathan, Chairperson, Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. M. Abhay Shankar, for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr.Manoranjini , Head of the Department, Department of Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr.Jinu Sophia for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and its learning resources, and I thank my family and friends for their unwavering support.

ABINASH G

ABSTRACT

Parking management in multi-level commercial complexes is still largely handled through registers, phone calls and spreadsheets. This leads to double-booked bays, no real-time view of occupancy, untracked payments and refunds, and manual reporting for administrators. This project designs and builds a Parking Slot Reservation & Management System on Zoho Creator, a low-code platform that provides hosted data storage, form builders, Deluge scripting, workflows, approvals, blueprints, dashboards and mobile access without separate server infrastructure.

The application covers seven forms for users, vehicles, parking slots across basement, ground and first floor, reservations, payments, cancellations and AI parking recommendation requests. It contains eight list reports, four pages including two dashboards and an AI assistant, ten Deluge scripts and eleven form-triggered workflows. Key business rules are enforced automatically: a validation script rejects overlapping bookings for the same slot, a Paid payment confirms the reservation, and scripts set the slot status, with the link between them tested in Chapter 6. A blueprint governs the reservation lifecycle across Pending, Confirmed, Cancelled and Refunded stages, an approval workflow routes reservations to an administrator, and a scheduled job sends a daily administrative summary.

Role-based profiles separate administrative, operator and user access. The review of the final export found ten defects, two of them rated high, and the eighteen test cases were traced statically against the code; they still need a live run. Deployment, measured performance and user acceptance results are still to be recorded. The system replaces manual steps with rule-driven automation and gives administrators live visibility of slots, reservations and payments.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Parking Management.

PROJECT AT A GLANCE

This one-page profile lets examiners verify the application quickly. Counts match the application export and Appendix A.

Table G.1: Project Profile

Attribute	Details
Project Title	Parking Slot Reservation & Management System
Application Domain	Facility Management / Smart Parking
Student Name & Register No.	ABINASH G – 240701007
Zoho Creator Edition	Free Trial
Application Name (Link Name)	Parking Slot Reservation & Management System
Published / Live URL	https://creator.zoho.in/userhome/ouerhgoifghej/admindashboard#/
Number of Forms / Subforms	6 / 0
Number of Reports (by type)	List: 8 Kanban: 0 Calendar: 0 Pivot/Chart: 0 (charts are placed on pages) Others: 0
Number of Pages / Dashboards	4 pages (2 dashboards, 2 AI pages)
Number of Workflows	Form: 11 Schedule: 1 Approval: 1 Blueprint: 1 (4 transitions) Batch: 0
Deluge Functions / Custom Actions	10 custom Deluge script blocks, plus 1 scheduled script
Integrations	Zoho Mail notifications (sendmail); Zoho Zia chat link. No external REST API.
Roles & Profiles Defined	Admin (User Role); User (User Role); profiles: Administrator, Admin_Full_Access, Write, Read, Limited_User_Access, Customer, Developer
Environments Used	Development

TABLE OF CONTENTS

ABSTRACT v
PROJECT AT A GLANCE vi
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 19
4.6 Access Control Design 20
IMPLEMENTATION 22
5.1 Application Setup 22
5.2 Forms and Data Capture 22
5.3 Reports 24

5.4 Pages and Dashboards 25
5.5 Deluge Scripting 26
5.6 Workflows and Automation 29
5.7 Integrations 33
5.8 User Management and Role-Based Access 33
5.9 AI and Analytics Features 34
5.10 Pro-Code Extension 35
5.11 Mobile and Portal Access 35
TESTING AND DEBUGGING 37
6.1 Testing Strategy 37
6.2 Test Cases 37
6.3 Debugging Deluge Scripts 38
6.4 Defect Log 39
6.5 User Acceptance Testing 39
DEPLOYMENT, SECURITY AND GOVERNANCE 41
7.1 Environments and Deployment Strategy 41
7.2 Performance Optimisation 41
7.3 Security Features 42
7.4 Governance and Administration 42
RESULTS AND DISCUSSION 44
8.1 Application Output 44
8.2 Achievement of Objectives 45
8.3 Comparison with the Existing System 45
8.4 Limitations of the Low-Code Approach 46
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 46
CONCLUSION AND FUTURE ENHANCEMENTS 47
9.1 Conclusion 47
9.2 Future Enhancements 47
9.3 Learning Outcomes and Career Relevance 47
REFERENCES 48
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 49
APPENDIX B – COURSE OUTCOME MAPPING 50
APPENDIX C – COMPLETE DELUGE SOURCE CODE 51
APPENDIX D – USER MANUAL 57
D.1 Accessing the Application 57
D.2 Instructions for Administrator 57
D.3 Instructions for User 57

APPENDIX E – EVALUATION RUBRIC 58
APPENDIX F – VIVA-VOCE RECORD SHEET 59

LIST OF TABLES

Table G.1: Project Profile vi
Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 9
Table 3.5: Use Case Description – Make Parking Reservation (UC-06) 11
Table 4.1: Entities and Relationships 13
Table 4.2: Field Specification – User_Details Form 15
Table 4.3: Field Specification – Vehicle_Details Form 15
Table 4.4: Field Specification – Parking_Slots Form 15
Table 4.5: Field Specification – Parking_Slot_Reservation Form 16
Table 4.6: Field Specification – Parking_Payment_Details Form 16
Table 4.7: Field Specification – Booking_Cancellation Form 16
Table 4.8: Field Specification – AI_Parking_Recommendations Form 17
Table 4.9: Role–Permission Matrix 21
Table 5.1: Reports Implemented 24
Table 5.2: Deluge Scripts Implemented 28
Table 5.3: Workflows Implemented 29
Table 5.4: Integrations Implemented 33
Table 6.1: Test Cases 37
Table 6.2: Defect Log 39
Table 6.3: User Acceptance Feedback 40
Table 7.1: Deployment Checklist 41
Table 7.2: Performance Optimisation Measures 42
Table 7.3: Security Controls Implemented 42
Table 8.1: Mapping of Objectives to Outcomes 45
Table 8.2: Existing System versus Proposed Application 45
Table A.1: Feature Coverage Checklist 49
Table B.1: Mapping of Course Units to Report Chapters 50
Table E.1: Evaluation Rubric for the Individual Project 58
Table F.1: Viva-Voce Questions and Responses 59
Table F.2: Marks Summary 59

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 5
Figure 3.1: Use Case Diagram of the Application 11
Figure 4.1: System Architecture of the Application 13
Figure 4.2: Entity–Relationship Diagram 14
Figure 4.3: Booking Lifecycle Workflow and Process Flow (intended design) 18
Figure 4.4: Navigation Structure of the Application 20
Figure 5.1: Zoho Creator Application List (top) and the Application Home with Navigation Sections (bottom) 23
Figure 5.2: Parking Slot Reservation Form in the Form Builder 23
Figure 5.3: Parking Slot Reservation Form in Live Mode 24
Figure 5.4: All_Parking_Slot_Reservations Report 25
Figure 5.5: Analytics Dashboard (Parking_Analytics_Dashboard page) 26
Figure 5.5A: Parking Dashboard: Parking Slot Status Overview 26
Figure 5.5B: Deluge Editor: DS-05 Booking Cancellation – Submission (as built) 29
Figure 5.6A: Form Workflows – Booking Cancellation and Update Reservation After Cancellation Approval 30
Figure 5.6B: Form Workflows – Parking Payment Details (Update Payment Status and five payment emails) 31
Figure 5.6C: Form Workflows – Parking Slot Reservation (Update Slot, Edited and Prevent Double Booking) 31
Figure 5.6D: Schedule – Daily Administrative Summary (31-Oct-2026, 09:00:00) 31
Figure 5.6E: Approvals – Reservation Approval on Parking Slot Reservation 32
Figure 5.6F: Reservation Lifecycle Blueprint Configuration 32
Figure 5.6G: Reservation Record at Blueprint Stage Confirmed, with Cancel and Refund Actions 32
Figure 5.7: Email Notification Output 33
Figure 5.8: Roles and Permission Profiles Configuration 34
Figure 5.9: AI Parking Assistant (top) and AI Parking Recommendation (bottom) 35
Figure 5.10: User Details Form in the Zoho Creator Mobile View 36
Figure 6.1: Deluge Debug Output / Log 38
Figure 7.1: Environments / Publish to Production 41
Figure 7.2: Audit Trail / Administration Settings 43
Figure 8.1: Scenario Step 1 – User creates a reservation (status Pending) 44
Figure 8.2: Scenario Step 2 – Operator records a Paid payment; reservation becomes Confirmed 44
Figure 8.3: Scenario Step 3 – Dashboard shows the slot as Occupied 45

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ER	Entity–Relationship
INR	Indian Rupee
IST	Indian Standard Time
KPI	Key Performance Indicator
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UPI	Unified Payments Interface

CHAPTER 1
INTRODUCTION

1.1 Background

Multi-level commercial parking facilities, such as shopping complexes, office campuses and hospitals, serve a continuous flow of two-wheeler and four-wheeler users. The facility modelled in this project has three parking areas (Basement, Ground Floor and First Floor) and separate bays for two-wheelers and four-wheelers. Such facilities usually allocate a bay at the gate, record the vehicle in a register or spreadsheet, and collect payment in cash or through UPI at exit. Reservations made in advance by phone or message are seldom linked to the actual bay occupancy, so staff rely on memory and physical checks to know which bays are free.

1.2 Problem Statement

In the manual process, the same bay can be promised to two users for overlapping times, a bay is not released when a booking is cancelled, and administrators have no live view of occupancy, revenue or refunds. The problem is to build one rule-driven system that prevents conflicting bookings, keeps slot status synchronised with reservation and payment events, and gives administrators live reporting under controlled access. The main pain points are:

Pain point 1 – Double booking. Overlapping reservations for the same bay are discovered only when two vehicles arrive at the gate.

Pain point 2 – Stale slot status. Bays stay marked as occupied after a cancellation or refund, and free bays are not known in real time.

Pain point 3 – Disconnected payments. Payments, failures and refunds are recorded separately from reservations, so reservation status and money do not agree.

Pain point 4 – Manual reporting. Daily counts of reservations, bays and payments are compiled by hand.

Pain point 5 – No access control. Every staff member sees and edits the same records.

1.3 Objectives

The objectives of the project are measurable and each maps to a module in Chapter 5 and a result in Chapter 8.

(O1) To design a normalised data model and seven forms for users, vehicles, parking slots, reservations, payments, cancellations and AI recommendation requests. (O2) To automate double-booking prevention, slot status, payment status and cancellation release using Deluge scripting and form workflows. (O3) To provide role-based dashboards and list reports for administrators and users. (O4) To integrate the application with Zoho Mail for payment and approval notifications and with Zia for an AI parking assistant. (O5) To deploy the application to production with an approval process, a blueprint-controlled lifecycle and a daily administrative summary.

1.4 Scope of the Project

In scope: the data model, forms with validation, list reports, dashboard and AI pages, form workflows, Deluge scripts, the Reservation Approval process, the Reservation Lifecycle blueprint, the Daily Administrative Summary schedule, role-based profiles, access through the Zoho Creator web and mobile apps, and the customer-portal profile. **Out of scope:** payment gateway integration (payments are recorded by operators), physical sensors and gate barriers, number-plate recognition, real-time vehicle detection, and billing based on actual exit time.

1.5 Justification for a Low-Code Approach

Zoho Creator provides hosting, a relational-style data store, a form builder, the Deluge scripting language, workflows, approvals, blueprints, dashboards and a mobile application in one platform [1], [2]. A parking operation of this kind needs all of these pieces, and a low-code approach lets the work be delivered within one semester with little infrastructure. Business rules are written as readable scripts, so an administrator can maintain the application without a separate development team. The trade-off is reduced control over the user interface and the runtime, which is discussed in Section 8.4.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The current process is carried out in five steps. (1) The vehicle arrives and the gate operator allocates a free bay from memory or a visual check. (2) The vehicle number and entry time are written in a register. (3) Advance bookings received by phone or message are noted separately and are not checked against the register. (4) At exit, the attendant calculates the charge by hand and records payment in cash or UPI. (5) At the end of the month, totals are compiled manually from the register and receipts.

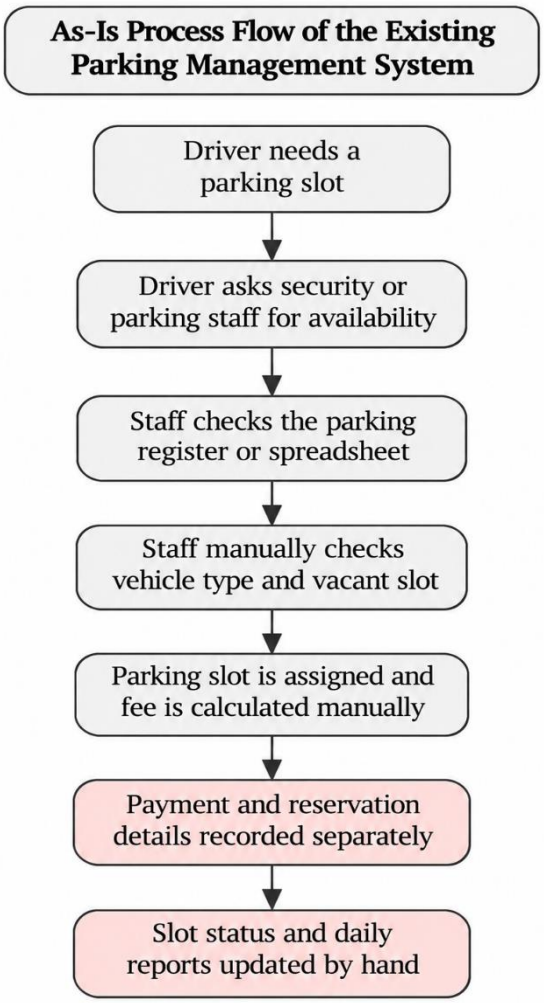


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

Limitation 1: no check exists for conflicting bookings, so double booking is found only at the gate. **Limitation 2:** cancellations do not free the bay in any record, so the bay appears occupied. **Limitation 3:** payments, failures and refunds are not linked to reservations. **Limitation 4:** there are no dashboards, and monthly figures take hours to compile. **Limitation 5:** there is no role-based access, so records can be changed by anyone with the register.

2.3 Low-Code versus Traditional Development

Table 2.1 compares a traditional, pro-code build of this application with the low-code build on Zoho Creator. The effort figures are estimates for a single student developer and are stated as such.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time (estimate)	8–10 weeks for a web and mobile build	2–3 weeks for the same scope
Skills required	Front-end, back-end, database and deployment skills	Form design, basic scripting (Deluge) and workflow logic
Hosting & infrastructure	Server, database and backup setup needed	Hosted and backed up by the platform
Mobile application	Separate native or hybrid app to build and publish	Same app is available in the Zoho Creator mobile app
Security & access control	Authentication and RBAC built manually	Profiles, permissions and PII flags configured in the platform
Maintenance & change	Code change, test and redeployment for each change	Form, workflow and report edits made in the builder
Customisation limits	Almost unlimited	Limited to platform components and Deluge scripting
Cost	Developer time, hosting and maintenance	Platform subscription

2.4 Literature Survey

Five sources were reviewed. They cover the definition of low-code platforms, comparison studies, the challenges of the approach, a process-automation case and the platform documentation used for this project.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Bock & Frank (2021) [3]	Low-Code Platform, Business & Information Systems Engineering	Discusses low-code platforms in terms of citizen developers and conceptual modelling	Supports the citizen-developer and model-driven design choices
2	Sahay et al. (2020) [4]	Supporting the understanding and comparison of low-code development platforms	Proposes a comparative framework and analyses eight low-code platforms by their features	Basis for the platform comparison in Section 2.3
3	Roķis & Kirikova (2022) [5]	Challenges of low-code/no-code software development: a literature review	Identifies challenges in the low-code development process and suggests ways to mitigate them	Informs the limitations (Section 8.4) and governance design (Section 7.4)
4	Waszkowski (2019) [6]	Low-code platform for automating business processes in manufacturing	Reports a case study using low-code for process automation	Shows workflow-based automation in an operational setting
5	Zoho Corporation (2026) [1]	Zoho Creator help documentation (Forms, Deluge, Workflows, Blueprint, Approvals)	Describes the platform features used	Reference for every feature in Chapter 5

2.5 Emerging Trends in Low-Code Development

Three trends shaped the design. Citizen development means that a staff member with domain knowledge, rather than a programmer, can keep the application up to date, which is why the rules are written in readable scripts and the screens are built with the form builder. Hyper-automation is reflected in the use of workflows, a blueprint and a scheduled summary so that routine actions happen without manual steps. AI-assisted building appears as the Zia assistant, which is included as an assistant page for parking questions and a request form for recommendations.

2.6 Proposed System

The proposed system is a Zoho Creator application with seven forms, eight list reports, four pages, eleven form workflows, a reservation approval process, a reservation lifecycle blueprint and a daily summary. It addresses each limitation in Section 2.2. Conflicting bookings are blocked by a validation script .

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Three roles use the system. Their profiles are defined in Chapter 4 (Table 4.9).

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Manages the facility. User Role = Admin. Profiles: Administrator, Admin_Full_Access	Manage slots, users and vehicles; approve reservations; view all dashboards and reports
Operator	Staff at the gate and accounts desk. Profile: Write	Create and update reservations and payments; record cancellations and refunds
User (Customer)	Registered vehicle owner. User Role = User. Profiles: Limited_User_Access, Customer	Register vehicles; book a slot; request cancellation; view own reservation status

3.2 Functional Requirements

Each requirement is written so that it can be tested. The FR identifiers are used again in the test cases in Chapter 6.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall register a user with a unique email, a valid Indian phone number, a role and an address	Administrator	High
FR-02	The system shall register a vehicle with a unique vehicle number linked to its owner	Administrator, User	High
FR-03	The system shall register a parking slot with a unique slot number, area, floor, vehicle type and hourly rate	Administrator	High
FR-04	The system shall create a reservation for a user, vehicle, slot, date, start time and end time	User, Operator	High
FR-05	The system shall reject a reservation whose end time is not later than its start time	System	High
FR-06	The system shall reject a reservation that overlaps an active reservation of the same slot on the same date	System	High
FR-07	The system shall mark a slot Occupied when its reservation is Confirmed, and Available when it is cancelled or refunded with no other confirmed overlap	System	High
FR-08	The system shall record payments with method and status; Paid confirms the reservation, Failed cancels a pending reservation, Refunded refunds it	Operator	High

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall send an email for each payment status change	System	Medium
FR-10	The system shall record a cancellation request; an approved cancellation cancels the reservation and frees the slot	User, Administrator	High
FR-11	The system shall route new reservations to an Administrator for approval and notify the submitter of the decision	Administrator	Medium
FR-12	The system shall send a daily administrative summary at 09:00 IST	System	Medium
FR-13	The system shall show dashboards with slot, reservation and payment KPIs and charts	Administrator	Medium
FR-14	The system shall provide an AI parking assistant page and capture recommendation requests	User	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Dashboards and list reports shall load within seconds for records; the target is to be measured during UAT
NFR-02	Security	Users shall access only the data permitted by their profile
NFR-03	Usability	The application shall be usable on desktop browsers and the Zoho Creator mobile app
NFR-04	Availability	The application shall be available whenever the Zoho Creator service is available
NFR-05	Maintainability	Business rules shall be kept in named scripts and workflows so that each rule can be found and changed
NFR-06	Auditability	Every record shall carry creation and modification details, and approval decisions shall be traceable

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Requirement
Client device	Any PC, laptop or smartphone with a modern web browser (Chrome or Edge)
Mobile access	Zoho Creator app for Android or iOS
Platform	Zoho Creator account (edition as in Project at a Glance)
Email service	Zoho Mail account linked to the Creator administrator (zoho.adminuserid)
AI assistant	Zia chat service (chat.zoho.com)

Category	Requirement
Development tools	Zoho Creator form builder, page builder, workflow and Deluge script editors

3.5 Use Case Model

Eighteen use cases are shown in Figure 3.1, grouped by the User and Admin actors. UC-06 (make a parking reservation) is described in detail in Table 3.5. UC-09 (request booking cancellation) and UC-15 (manage payments) follow the same pattern.

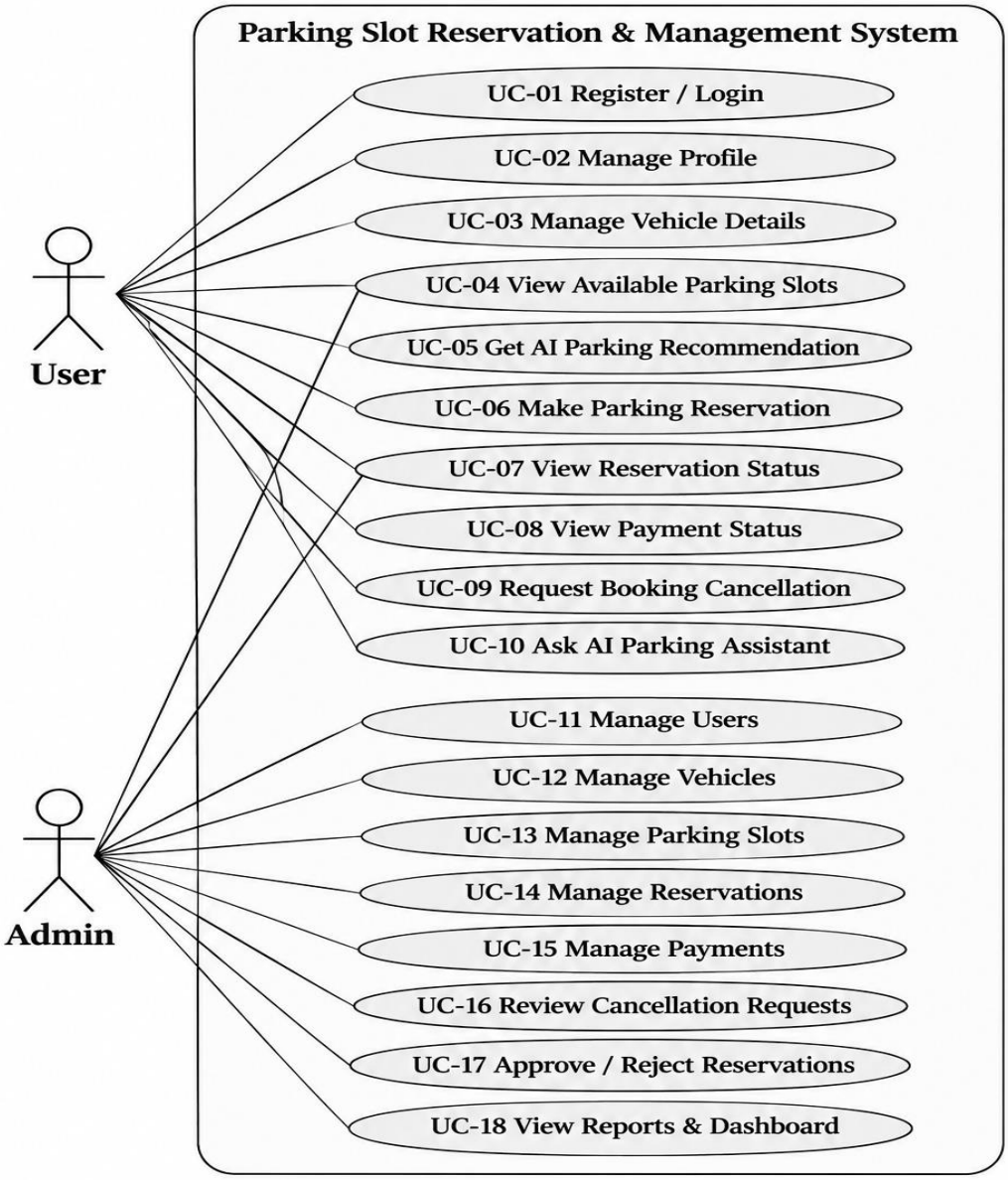


Figure 3.1: Use Case Diagram of the Application

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows four layers. The access and presentation layer covers the Zoho Creator web browser and mobile app. The application and business logic layer holds the forms, workflows, approval, blueprint, reports, dashboard and AI features. The data layer holds the six data tables, and the external services layer holds Zoho Zia and email notifications. Users never change data directly in a report; they submit forms, and the logic layer then updates the related records. This keeps rule enforcement in one place.

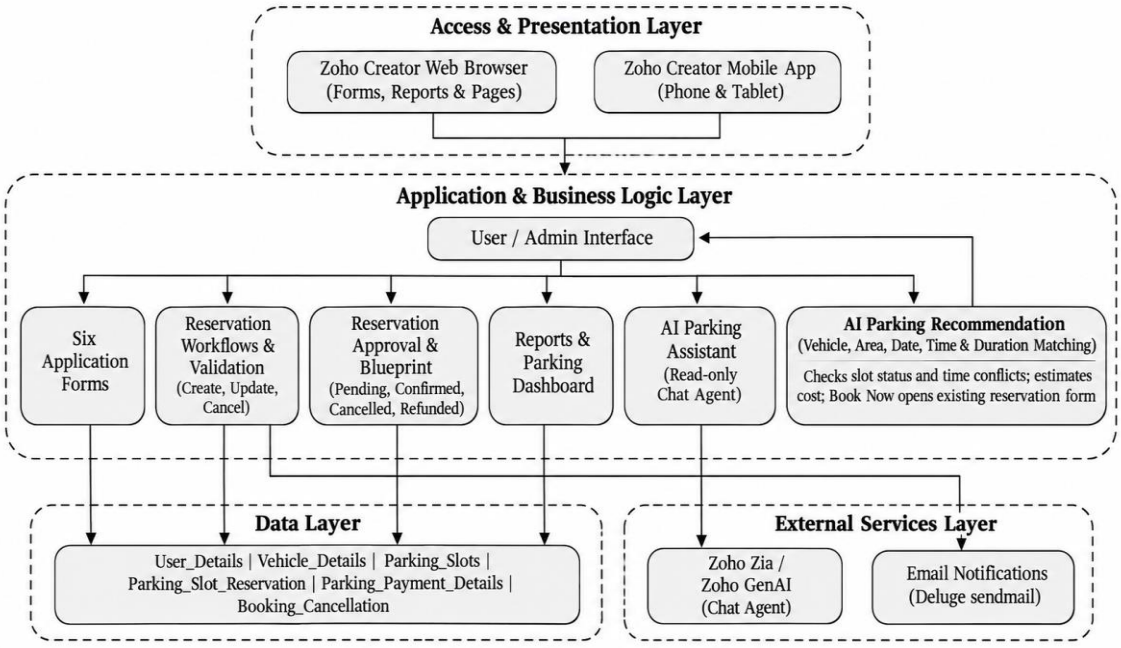


Figure 4.1: System Architecture of the Application

4.2 Data Model

The data model is normalised so that each fact is stored once. Vehicles are separate from users because one user can own several vehicles. Payments are separate from reservations because one reservation can have several payment attempts (for example, a failed UPI payment followed by a successful one). Cancellations are separate for the same reason and so that a reservation keeps its history.

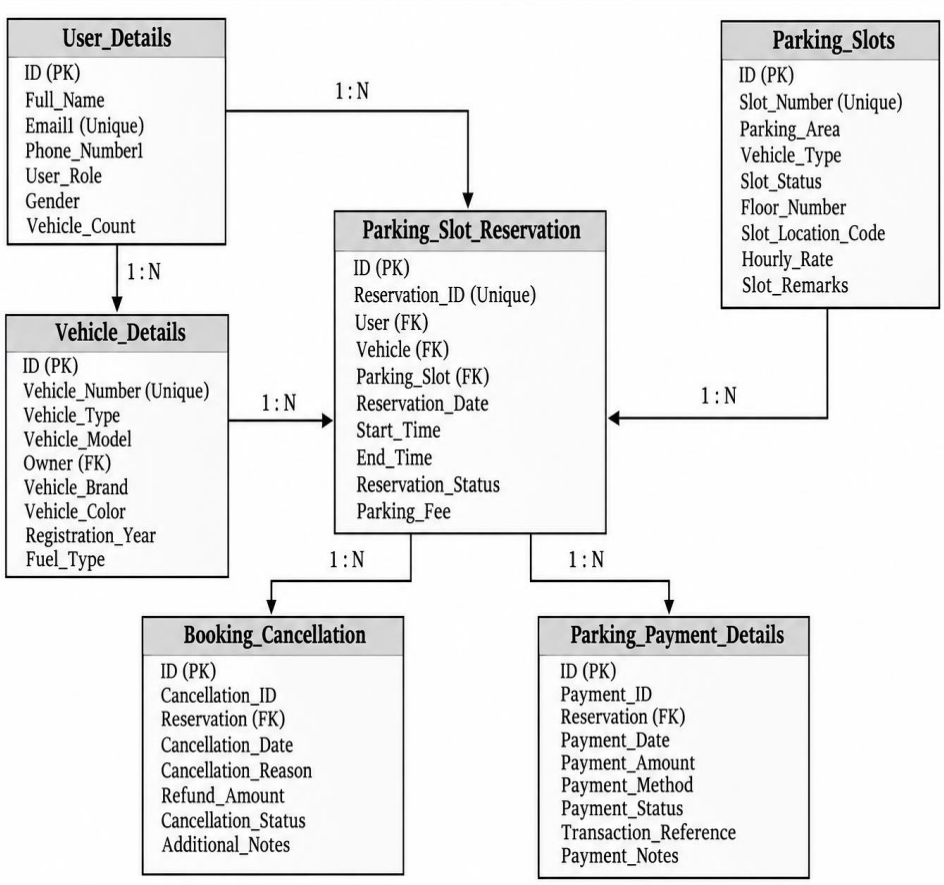


Figure 4.2: Entity–Relationship Diagram

4.3 Form Design

Each form is laid out in one section with a row and column grid. Required fields are marked as mandatory, identifiers are marked unique, and picklists restrict input to the allowed values, for example Payment Method (UPI, Cash, Debit Card, Credit Card, Net Banking) and Cancellation Reason (Change of Plans, Wrong Slot Selected, Vehicle Changed, Duplicate Booking, Other). Help text is attached to the important fields. Each form has an Add action (Submit) and an Edit action (Update and Cancel).

Tables 4.2 to 4.8 give the field specification of each form, taken from the application export. Mandatory and unique rules are the ones enforced by the platform; the options column lists the picklist values or the lookup target. Table 4.5 covers the reservation form, which carries most of the business rules.

Table 4.2: Field Specification – User_Details Form

Field	Type	Mandatory	Unique	Options / Rule
Full Name	Name	Yes	No	-
Email	Email	Yes	Yes	max 80 characters
Phone Number	Phone	Yes	No	-
User Role	Drop Down	Yes	No	Options: Admin, User; max 100 characters
Date of Birth	Date	No	No	-
Gender	Drop Down	Yes	No	Options: Male, Female, Prefer not to say; max 100 characters
Address Line 1	Address	No	No	-
Vehicle Count	Number	No	No	-

Table 4.3: Field Specification – Vehicle_Details Form

Field	Type	Mandatory	Unique	Options / Rule
Vehicle Number	Single Line	Yes	Yes	-
Vehicle Type	Drop Down	Yes	No	Options: Two Wheeler, Four Wheeler; max 100 characters
Vehicle Model	Single Line	No	No	-
Owner	Drop Down	Yes	No	Lookup to User_Details
Vehicle Brand	Single Line	No	No	max 50 characters
Vehicle Color	Single Line	No	No	max 30 characters
Registration Year	Number	No	No	max 4 characters
Fuel Type	Drop Down	No	No	Options: Petrol, Diesel, Electric, Hybrid, CNG; max 100 characters

Table 4.4: Field Specification – Parking_Slots Form

Field	Type	Mandatory	Unique	Options / Rule
Slot Number	Single Line	Yes	Yes	-
Parking Area	Drop Down	Yes	No	Options: Basement, Ground Floor, First Floor; max 100 characters
Vehicle Type	Drop Down	Yes	No	Options: Two Wheeler, Four Wheeler; max 100 characters
Slot Status	Drop Down	Yes	No	Options: Available, Occupied; max 100 characters
Floor Number	Number	Yes	No	max 2 characters
Slot Location Code	Single Line	No	No	max 30 characters
Hourly Rate	Currency (INR)	Yes	No	max 8 characters

Field	Type	Mandatory	Unique	Options / Rule
Slot Remarks	Multi Line	No	No	-

Table 4.5: Field Specification – Parking_Slot_Reservation Form

Field	Type	Mandatory	Unique	Options / Rule
Reservation ID	Single Line	Yes	Yes	max 30 characters
User	Drop Down	Yes	No	Lookup to User_Details
Vehicle	Drop Down	Yes	No	Lookup to Vehicle_Details
Parking Slot	Drop Down	Yes	No	Lookup to Parking_Slots
Reservation Date	Date	Yes	No	-
Start Time	Time	Yes	No	-
End Time	Time	Yes	No	-
Reservation Status	Drop Down	Yes	No	Options: Pending, Confirmed, Cancelled, Refunded; max 100 characters
Parking Fee	Currency (INR)	No	No	max 8 characters

Table 4.6: Field Specification – Parking_Payment_Details Form

Field	Type	Mandatory	Unique	Options / Rule
Payment ID	Single Line	Yes	Yes	max 30 characters
Reservation	Drop Down	Yes	No	Lookup to Parking_Slot_Reservation
Payment Date	Date	Yes	No	-
Payment Amount	Currency (INR)	Yes	No	max 8 characters
Payment Method	Drop Down	Yes	No	Options: UPI, Cash, Debit Card, Credit Card, Net Banking; max 100 characters
Payment Status	Drop Down	Yes	No	Options: Pending, Paid, Failed, Refunded; max 100 characters
Transaction Reference	Single Line	No	No	max 100 characters
Payment Notes	Multi Line	No	No	-

Table 4.7: Field Specification – Booking_Cancellation Form

Field	Type	Mandatory	Unique	Options / Rule
Cancellation ID	Single Line	Yes	Yes	max 30 characters
Reservation	Drop Down	Yes	No	Lookup to Parking_Slot_Reservation
Cancellation Date	Date	Yes	No	-

Field	Type	Mandatory	Unique	Options / Rule
Cancellation Reason	Drop Down	Yes	No	Options: Change of Plans, Wrong Slot Selected, Vehicle Changed, Duplicate Booking, Other; max 100 characters
Refund Amount	Currency (INR)	No	No	-
Cancellation Status	Drop Down	Yes	No	Options: Pending, Approved, Rejected; max 100 characters
Additional Notes	Multi Line	No	No	-

Table 4.8: Field Specification – AI_Parking_Recommendations Form

Field	Type	Mandatory	Unique	Options / Rule
Preferred Location	Single Line	No	No	-
Booking Date	Date	No	No	-
Vehicle Type	Single Line	No	No	-
Booking Time	Time	No	No	-
Recommended Slot	Single Line	No	No	-
Duration Hours	Number	No	No	-

Name parts (prefix, first name, last name, suffix) and address parts (line 2, city, state, postal code, country, latitude and longitude) are optional sub-fields of Full Name and Address. The Submit, Update and Cancel buttons are omitted from the tables.

4.4 Process and Workflow Design

The reservation lifecycle is controlled by the Reservation Lifecycle blueprint. It has four stages (Pending, Confirmed, Cancelled and Refunded) and four transitions, each with a short script that updates Reservation_Status. A reservation starts as Pending. A Paid payment or the Confirm transition moves it to Confirmed. Cancellation moves it to Cancelled from either Pending or Confirmed. A refund moves a Confirmed reservation to Refunded. Figure 4.3 shows the lifecycle. The approval process routes the reservation to an Administrator but does not change its status in the current export (defect D04).

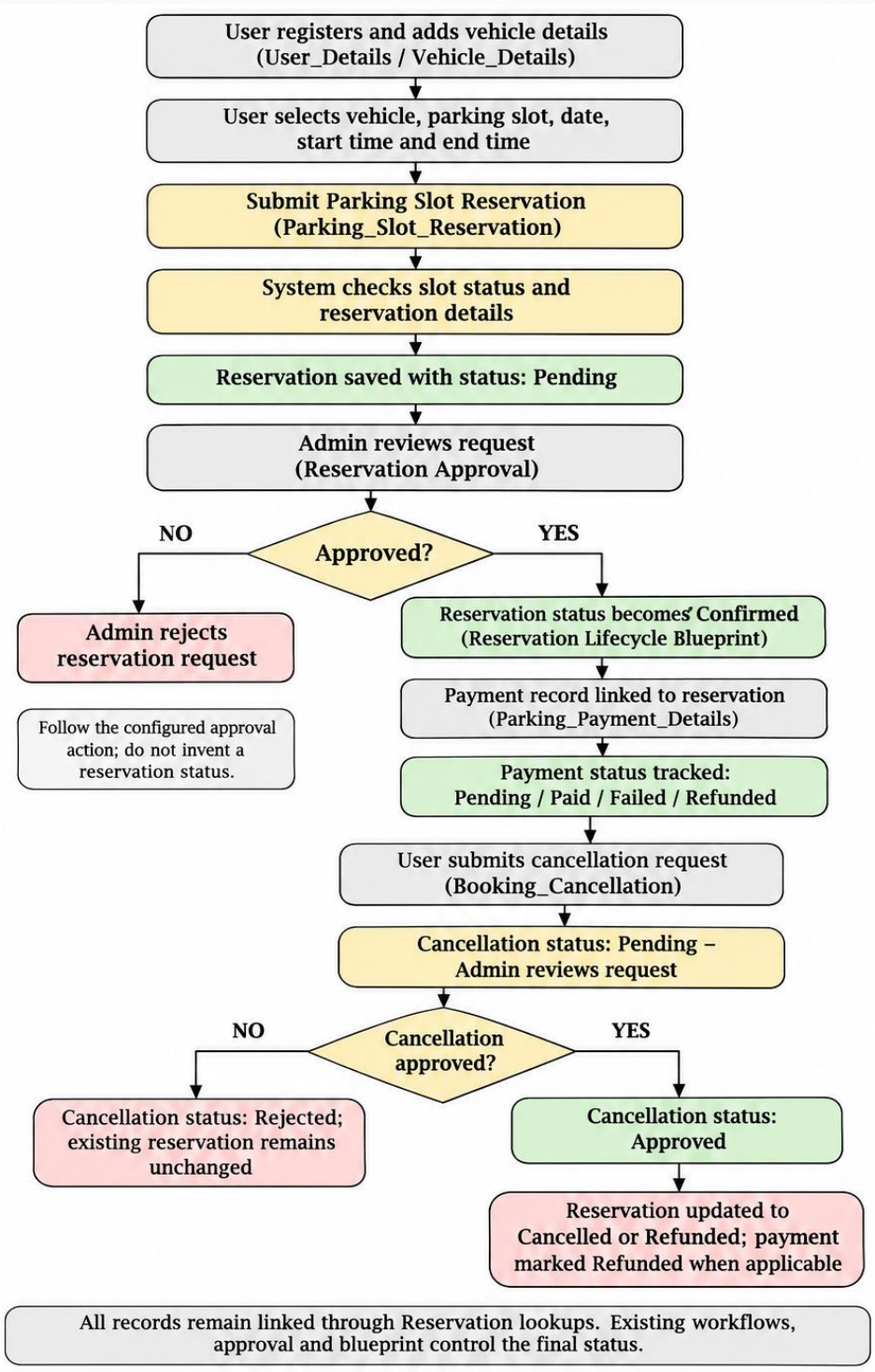


Figure 4.3: Booking Lifecycle Workflow and Process Flow (intended design)

Slot status is changed only by the scripts, never by the user directly. A confirmed reservation marks its slot Occupied. A cancelled or refunded reservation releases the slot only when no other confirmed reservation overlaps the same time on the same date.

4.5 User Interface and Navigation Design

Figure 4.4 groups the menu into Forms (the data forms), Reports (the list reports), AI Features (the AI assistant, the recommendation page and its list), Approvals (Reservation Approval), Blueprint and Workflows (the reservation lifecycle, the form workflows and the daily administrative summary) and Dashboard (the Parking Dashboard). The profiles in Table 4.9 decide which groups each user can open.

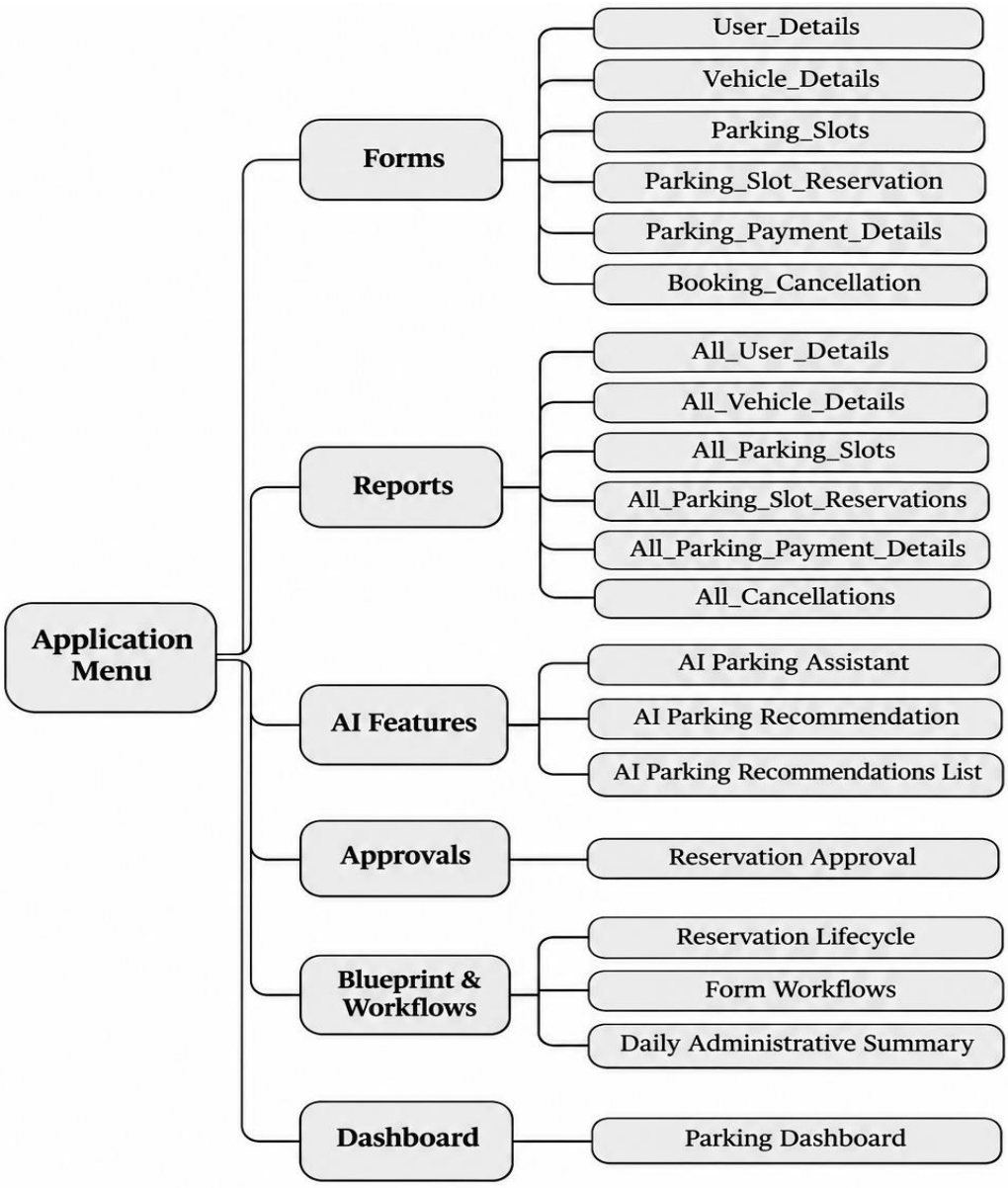


Figure 4.4: Navigation Structure of the Application

4.6 Access Control Design

Access is controlled by profiles. Table 4.9 shows the permissions defined in the application. The Administrator and Admin_Full_Access profiles manage all forms, records, cancellations, payments and dashboards. Write and Read give broad access to every component. Limited_User_Access allows a user to create records and view reservations, cancellations, vehicles and slots, but not to change them. The Customer profile is a portal profile that allows adding and viewing only. Personal fields (email and phone) are flagged as personal data, so that they are handled as PII.

Table 4.9: Role–Permission Matrix

Profile	Type	Key Permissions
Administrator	Users & Permissions	Chat, Predefined, API access, PII and ePHI access on all components
Admin_Full_Access	Users & Permissions	Full management of forms, records, cancellations, payments and the dashboard; API and PII access
Write	Users & Permissions	Write access to all components
Read	Users & Permissions	Read access to all components
Limited_User_Access	Users & Permissions	Create and tab on user records; view-only on cancellations, reservations, vehicles and slots
Customer	Customer portal	Add and view only
Developer	Developer	Predefined and API access; PII access; no chat

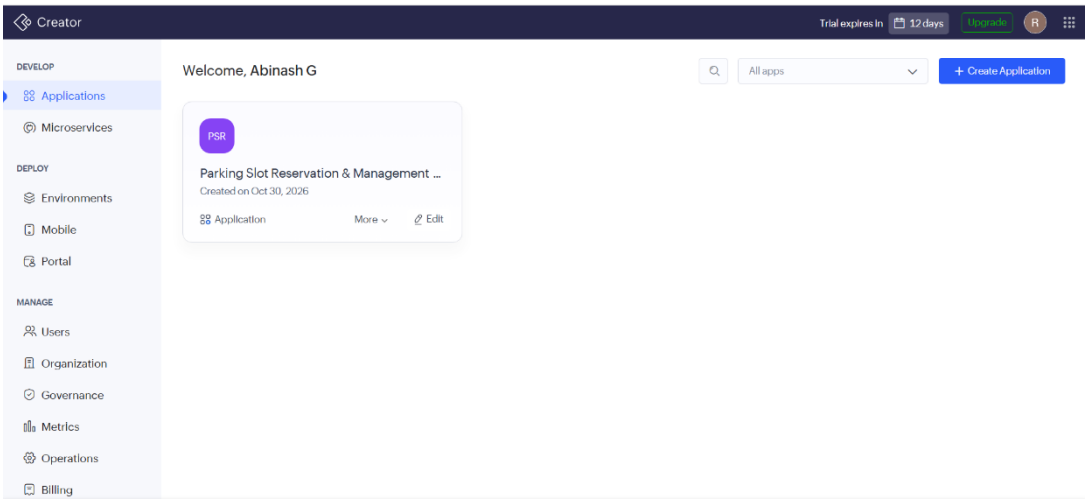
CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application is named Parking Slot Reservation & Management System. Dates use the format dd-MMM-yyyy, the time zone is Asia/Kolkata and the clock uses the 24-hour format. The application has a theme and a language selection under App Preferences. The navigation sections and their icons are set as described in Section 4.5. The Integrations entry is used to connect Zoho services.

5.2 Forms and Data Capture

Seven forms capture the data. User_Details records the personal and contact details, the role, date of birth, gender, address and vehicle count. Vehicle_Details records each vehicle with its owner, type (two-wheeler or four-wheeler), brand, model, colour, registration year and fuel type. Parking_Slots records each bay with its area, floor, location code, vehicle type, status (Available or Occupied), hourly rate and remarks. Parking_Slot_Reservation records the booking, as specified in Table 4.5. Parking_Payment_Details records each payment attempt with its method, status, amount, transaction reference and notes. Booking_Cancellation records each cancellation with its reason, refund amount and status (Pending, Approved or Rejected). AI_Parking_Recommendations captures the preferred location, booking date and time, vehicle type, duration and recommended slot.



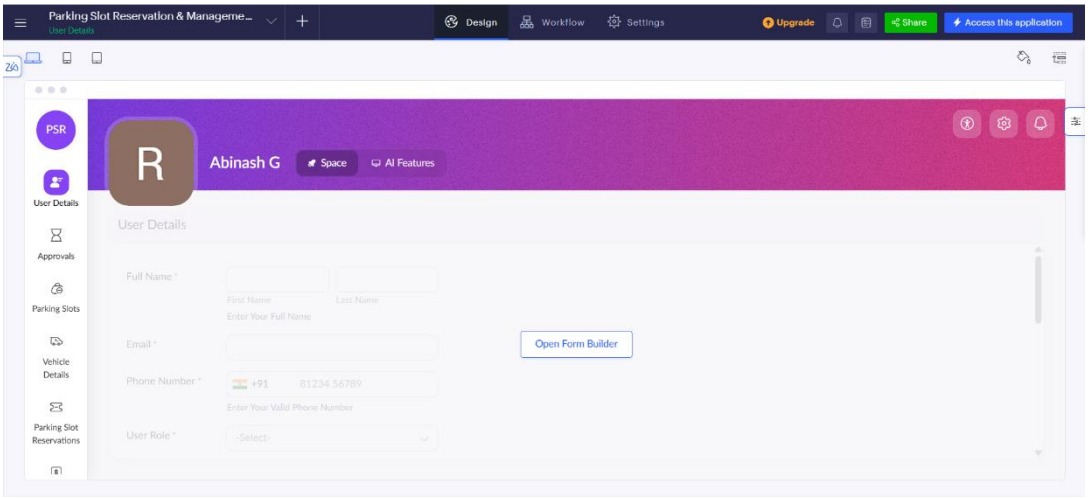


Figure 5.1: Zoho Creator Application

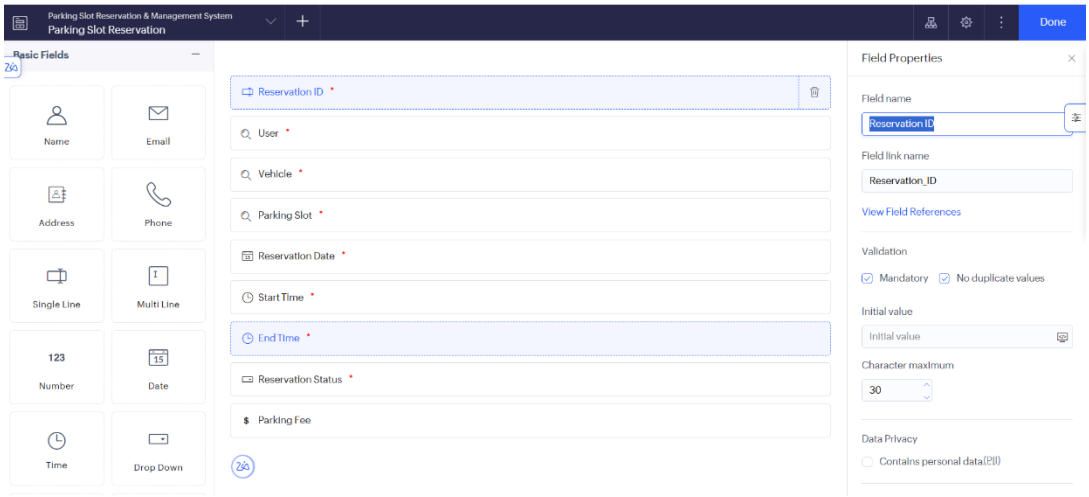


Figure 5.2: Parking Slot Reservation Form in the Form Builder

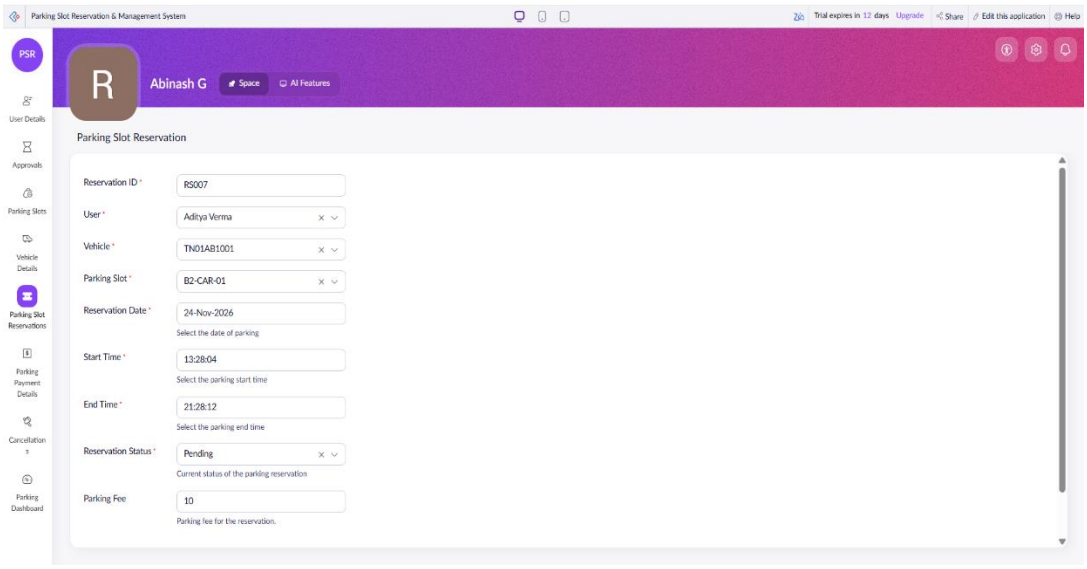


Figure 5.3: Parking Slot Reservation Form in Live Mode

5.3 Reports

Eight list reports present the records to users. They are summarised in Table 5.1. Each report has view, edit and delete permissions as set in its profile, and the reports are placed in the Reports section and on the dashboards.

Table 5.1: Reports Implemented

Report	Source Form	Purpose
All_User_Details	User_Details	List of users with contact details and role
All_User_Details1	User_Details	Second copy of the user list used in the Reports section (to be removed or renamed; see defect D09 in Table 6.2)
All_Vehicle_Details	Vehicle_Details	List of vehicles with owner and specification
All_Parking_Slots	Parking_Slots	List of slots with area, status and hourly rate
All_Parking_Slot_Reservations	Parking_Slot_Reservation	Reservations with slot, time and fee
All_Parking_Payment_Details	Parking_Payment_Details	Payments with method, status and amount
All_Cancellations	Booking_Cancellation	Cancellations with reason, refund and status
AI_Parking_Recommendations_List	AI_Parking_Recommendations	Recommendation requests captured from the AI page

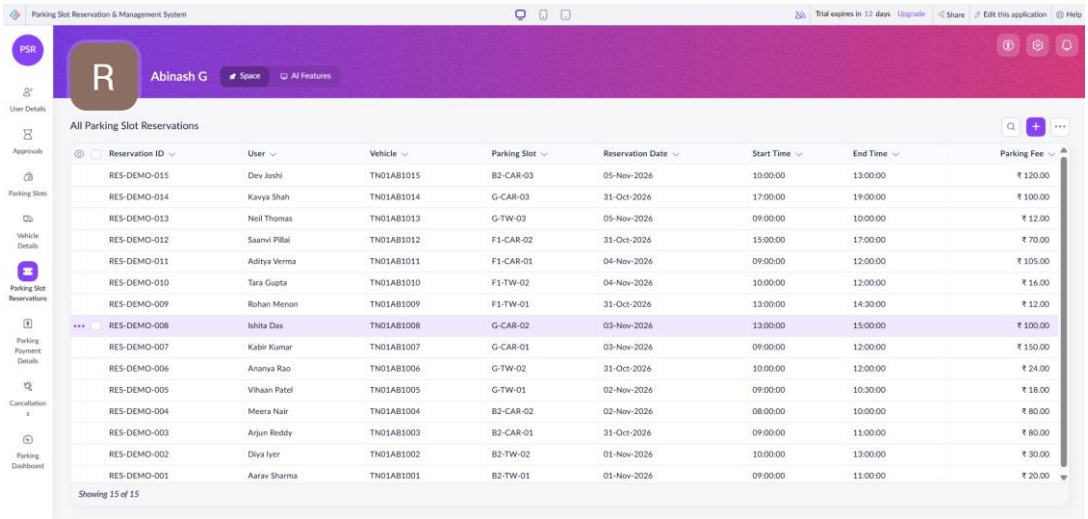


Figure 5.4: All_Parking_Slot_Reservations Report

5.4 Pages and Dashboards

Parking Dashboard. This page is headed “Parking Management”. It shows four KPI cards (total slots, available slots, occupied slots and pending reservations), a donut chart of slot status, a bar chart of reservation status, a pie chart of payment status, and a slot report with a search bar. **Parking Analytics Dashboard.** This page shows reservation status as a column chart, payment status as a line chart and slot status as an area chart, with donut and pie charts for the status distribution. **AI Parking Assistant.** This page introduces the Zia-based assistant for parking questions, with a button that opens the chat service. **AI Parking Recommendation.** This page holds the recommendation request form, a list of past requests and summary cards.

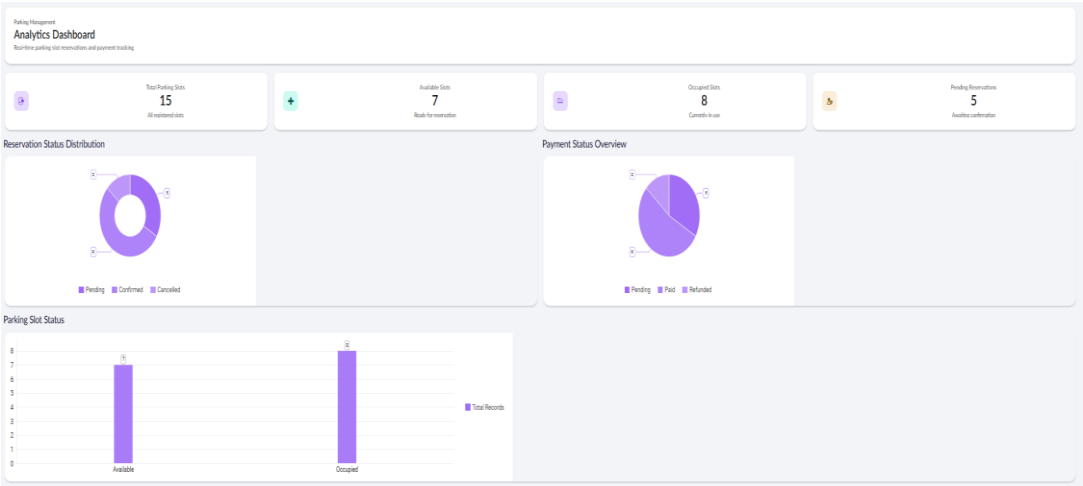


Figure 5.5: Analytics Dashboard

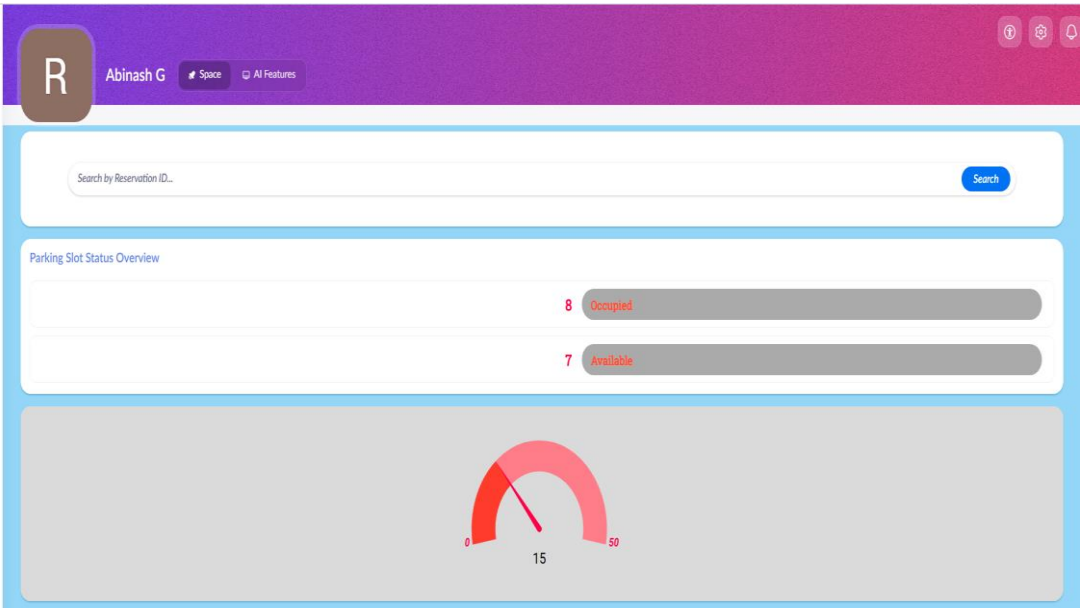


Figure 5.5A: Parking Dashboard: Parking Slot Status Overview

5.5 Deluge Scripting

Six form-triggered scripts, four blueprint transition scripts and one scheduled script implement the rules. Table 5.2 lists them and Appendix C reproduces the full source. The most important rules are summarised here.

Double-booking prevention. The Prevent Double Booking script runs on validation when a reservation is added or edited. It first rejects a reservation whose end time is not later than its start time. It then searches for reservations of the same slot on the same date that are not Cancelled or Refunded and whose time interval overlaps the new one, ignoring the record being edited. If one is found, the submission is cancelled with an alert.

Code 5.1: Prevent Double Booking validation (DS-01, excerpt)

```
// 1. Validate start and end time
if(input.Start_Time != null && input.End_Time != null)
{
    if(input.Start_Time >= input.End_Time)
    {
        alert "End Time must be later than Start Time.";
        cancel submit;
    }
}
// 2. Check overlapping reservations
if(input.Parking_Slot != null && input.Reservation_Date != null &&
input.Start_Time != null && input.End_Time != null)
{
    conflictingReservations = Parking_Slot_Reservation[Parking_Slot ==
input.Parking_Slot && Reservation_Date == input.Reservation_Date &&
Reservation_Status != "Cancelled" && Reservation_Status != "Refunded" &&
Start_Time < input.End_Time && End_Time > input.Start_Time];
    for each  reservation in conflictingReservations
    {
        if(reservation.ID != input.ID)
        {
            alert "This parking slot is already reserved for the selected
time. Please choose another slot or time.";
            cancel submit;
        }
    }
}
```

Payment status. The Update Payment Status script runs when a payment is added. A Paid payment confirms the reservation. A Failed payment cancels the reservation only if it is still Pending. A Refunded payment sets the reservation to Refunded. A Pending payment sets the reservation to Pending unless it is already Confirmed. The script leaves the reservation unchanged for any other value.

Slot status. The two slot scripts are meant to keep Parking_Slots in step with reservations. The add version marks a Confirmed reservation's slot Occupied, and releases a Cancelled or Refunded reservation's slot only if no other confirmed reservation overlaps it in time. The edit version uses a simpler test, which looks for any other confirmed reservation of the slot on that date (defect D08). When a cancellation is approved, the Booking Cancellation scripts set the reservation to Cancelled and the slot to Available. The edit version skips the update if the reservation is already Cancelled, so a repeated update is safe. Neither version checks for other confirmed bookings on the slot, which is recorded as defect D06.

Code 5.2: Update Payment Status (DS-04, excerpt)

```
resId = "";
if(input.Reservation != null)
{
    resRec = Parking_Slot_Reservation[ID == input.Reservation];
    resId = resRec.Reservation_ID;
    currentStatus = resRec.Reservation_Status;
    if(input.Payment_Status == "Paid")
    {
        resRec.Reservation_Status="Confirmed";
        info "Payment is recorded as Paid for Reservation ID: " + resId;
    }
    else if(input.Payment_Status == "Failed")
    {
        if(currentStatus == "Pending")
        {
            resRec.Reservation_Status="Cancelled";
        }
        info "Payment has been recorded as Failed for Reservation ID: " +
resId;
    }
    else if(input.Payment_Status == "Refunded")
    {
        resRec.Reservation_Status="Refunded";
    ... (full script in Appendix C)
```

Table 5.2: Deluge Scripts Implemented

ID	Script	Trigger	Purpose
DS-01	Prevent Double Booking	Parking_Slot_Reservation, on validate	Rejects invalid times and overlapping reservations
DS-02	Update Slot After Reservation	Parking_Slot_Reservation, on add	Sets slot status from the new reservation status
DS-03	Update Slot After Reservation Edited	Parking_Slot_Reservation, on edit	Sets slot status after a reservation is edited
DS-04	Update Payment Status	Parking_Payment_Details, on add	Maps payment status to reservation status
DS-05	Booking Cancellation – Submission	Booking_Cancellation, on add	Cancels the reservation and frees the slot when Approved

ID	Script	Trigger	Purpose
DS-06	Update Reservation After Cancellation Approval	Booking_Cancellation, on edit	Same release on approval; skipped if already Cancelled
DS-07	Cancel Reservation	Blueprint: Pending to Cancelled	Sets Reservation_Status to Cancelled
DS-08	Confirm Reservation	Blueprint: Pending to Confirmed	Sets Reservation_Status to Confirmed
DS-09	Cancel Confirmed Reservation	Blueprint: Confirmed to Cancelled	Sets Reservation_Status to Cancelled
DS-10	Refund Reservation	Blueprint: Confirmed to Refunded	Sets Reservation_Status to Refunded
DS-11	Daily Administrative Summary	Schedule, daily 09:00 IST (on load)	Counts reservations, slots and payments and emails the summary

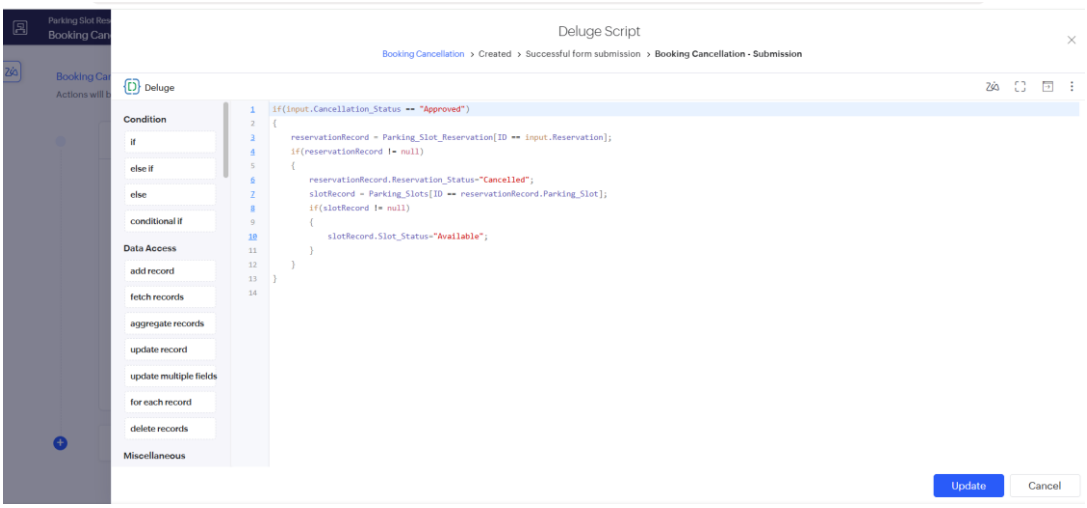


Figure 5.5B: Deluge Editor: DS-05 Booking Cancellation – Submission (as built)

5.6 Workflows and Automation

Eleven form workflows connect the forms to the scripts and to the emails. Five of them send a payment status email for Payment Confirmation (on add or edit), Paid, Failed, Pending and Refunded payments. The Pending email is sent to the administrator account. The other payment emails are addressed to the Zoho account owner (zoho.loginuserid), not to the registered vehicle owner, so customer notification is recorded as defect D07. The Reservation Approval process routes each new reservation to a Level 1 approver with the Admin role and sends an approval or rejection email to the submitter. The Daily Administrative Summary runs every day at 09:00 Asia/Kolkata and emails the counts of reservations, slots and payments to the

administrator. Table 5.3 summarises the workflows. Figures 5.6A to 5.6E show the workflow and schedule screens, and Figure 5.6F shows the blueprint configuration.

Table 5.3: Workflows Implemented

Workflow	Form / Component	Event	Action
Booking Cancellation – Submission	Booking_Cancellation	On add	Script (cancel and release slot if Approved)
Update Reservation After Cancellation Approval	Booking_Cancellation	On edit	Script (cancel and release slot)
Update Slot After Reservation	Parking_Slot_Reservation	On add	Script (slot status)
Update Slot After Reservation Edited	Parking_Slot_Reservation	On edit	Script (slot status)
Prevent Double Booking	Parking_Slot_Reservation	On validate	Script (time and overlap checks)
Update Payment Status	Parking_Payment_Details	On add	Script (reservation status)
Payment Confirmation Email	Parking_Payment_Details	On add or edit	Email with payment details
Paid Payment Email	Parking_Payment_Details	On add or edit, if Paid	Email confirming payment
Failed Payment Email	Parking_Payment_Details	On add or edit, if Failed	Email reporting failure
Pending Payment Email	Parking_Payment_Details	On add or edit, if Pending	Email to the administrator
Refunded Payment Email	Parking_Payment_Details	On add or edit, if Refunded	Email confirming refund
Daily Administrative Summary	Schedule	Daily 09:00 IST	Summary email of counts
Reservation Approval	Parking_Slot_Reservation	Approval, Level 1 (Admin)	Approve or reject; email the submitter
Reservation Lifecycle	Parking_Slot_Reservation	Blueprint transitions	Four transition scripts

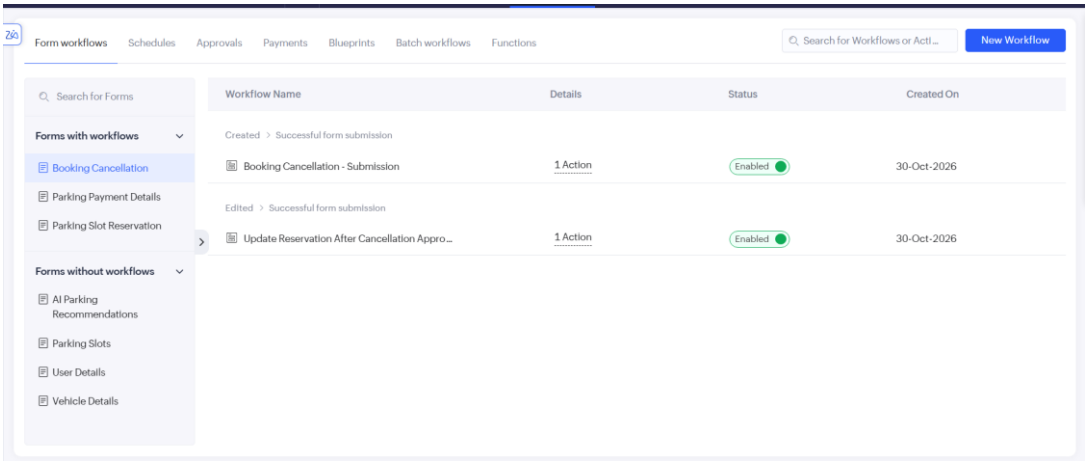


Figure 5.6A: Form Workflows – Booking Cancellation and Update Reservation After Cancellation Approval

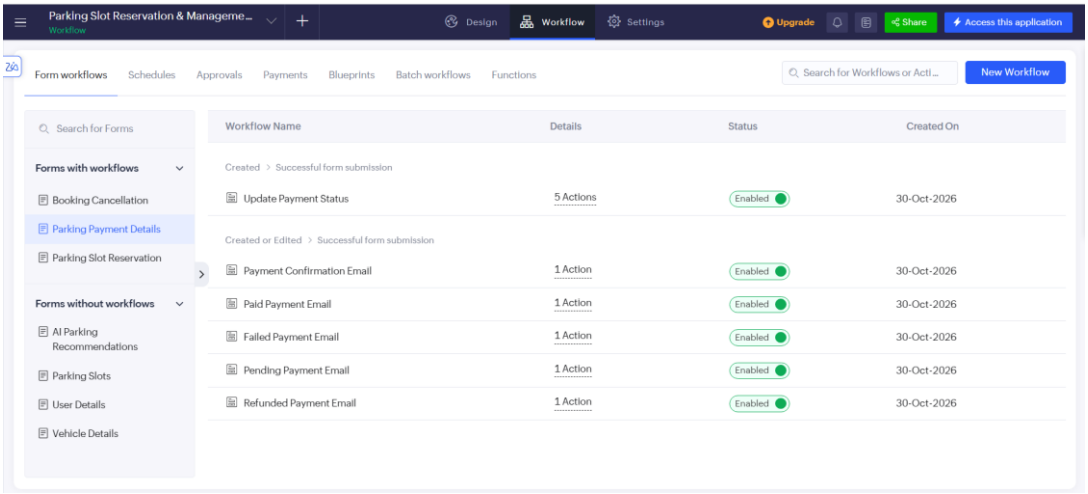


Figure 5.6B: Form Workflows – Parking Payment Details (Update Payment Status and five payment emails)

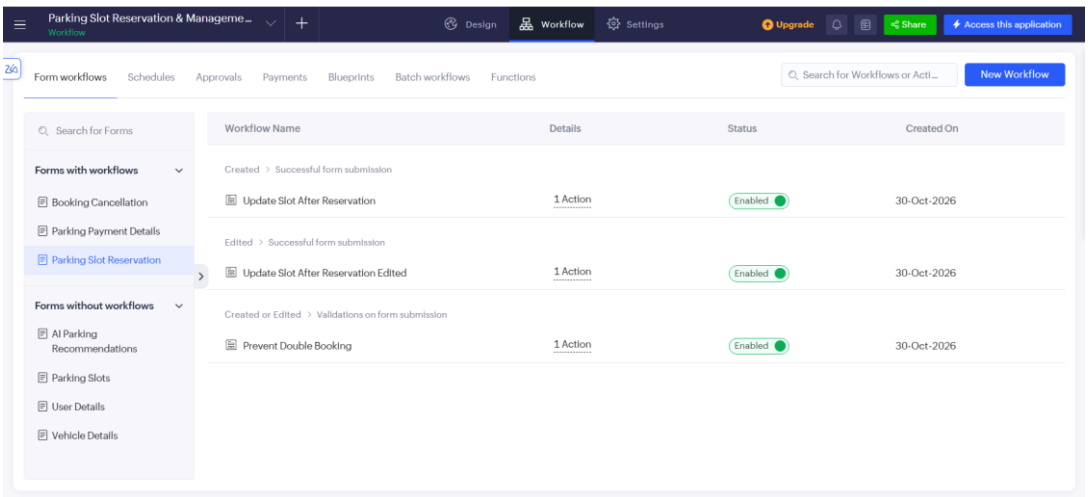


Figure 5.6C: Form Workflows – Parking Slot Reservation (Update Slot, Edited and Prevent Double Booking)

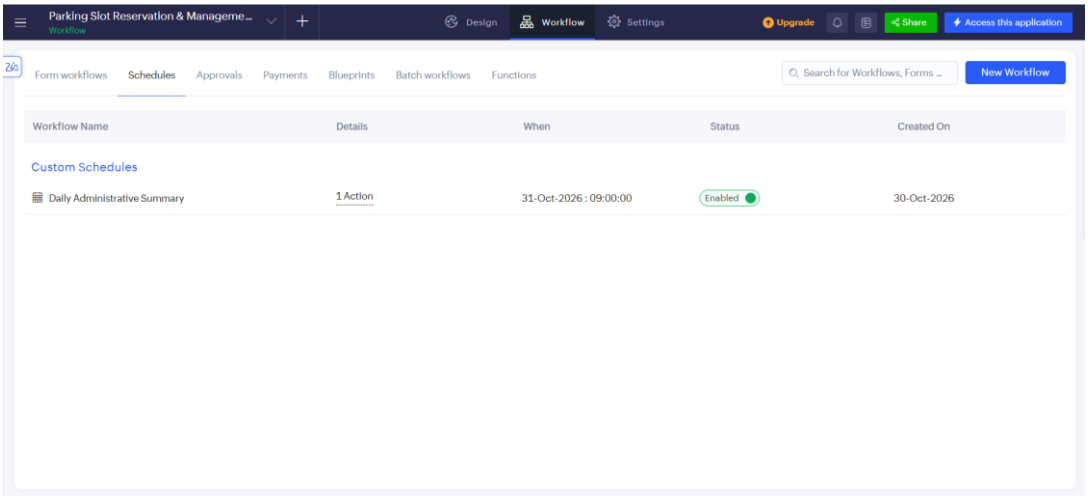


Figure 5.6D: Schedule – Daily Administrative Summary (31-Oct-2026, 09:00:00)

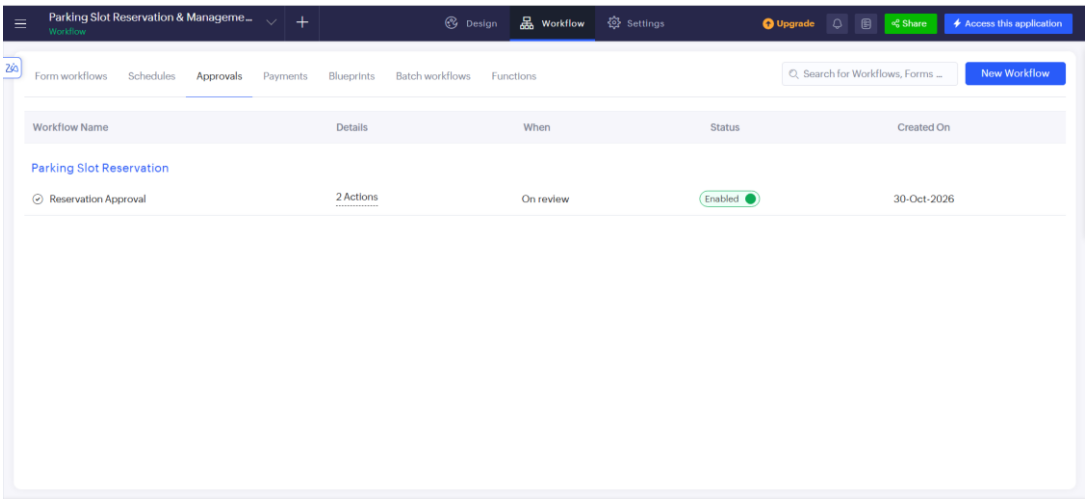


Figure 5.6E: Approvals – Reservation Approval on Parking Slot Reservation

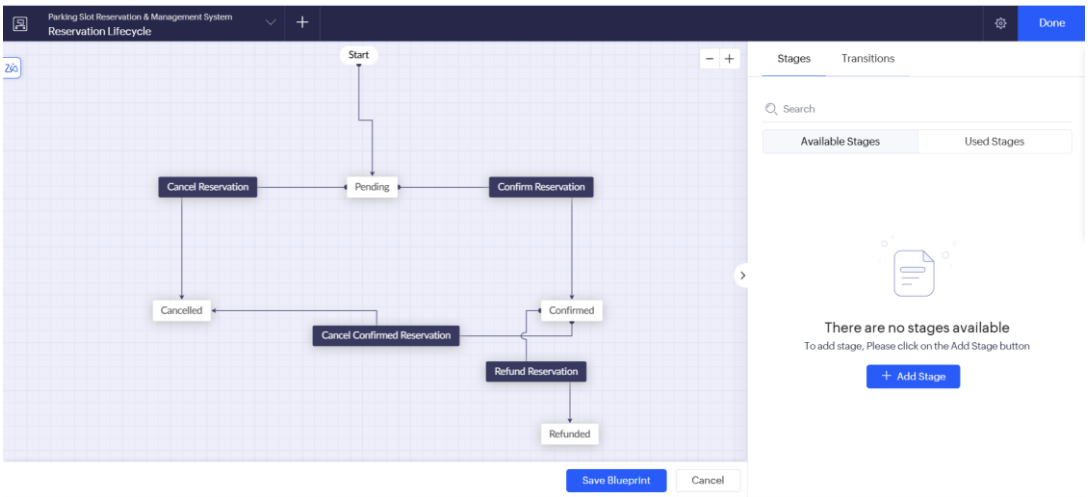


Figure 5.6F: Reservation Lifecycle Blueprint Configuration

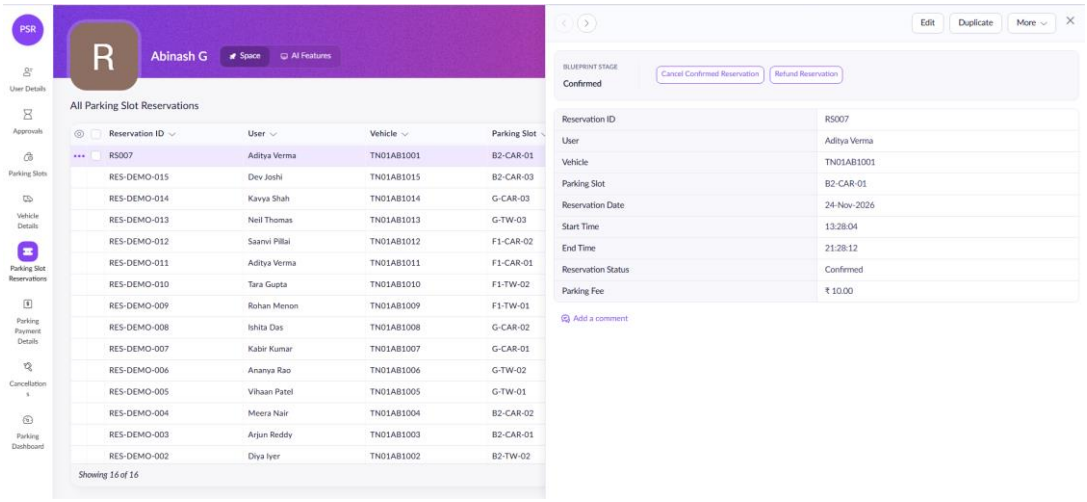


Figure 5.6G: Reservation Record at Blueprint Stage Confirmed, with Cancel and Refund Actions

5.7 Integrations

The application uses two integrations and does not call an external REST API. Email notifications are sent through Zoho Mail using the administrator account of the application. The AI assistant page links to the Zia chat service. A payment gateway and a REST API integration are outside the scope in Section 1.4, because payments are recorded by operators and the application needs no external data source. Table 5.4 summarises the status of each integration.

Table 5.4: Integrations Implemented

Integration	Type	Purpose	Status
Zoho Mail (sendmail)	In-platform email	Payment, approval and summary emails	Implemented
Zia chat (chat.zoho.com)	Link from AI page	Parking help and questions	Implemented as a link
Payment gateway (UPI or card)	External API	Automatic payment capture	Not implemented (out of scope)
External REST API	Web service	Live external data	Not implemented

5.8 User Management and Role-Based Access

Users are created in User_Details with a User Role of Admin or User. The profiles in Table 4.9 are then assigned to the users. Administrators see all records and can approve reservations, while users see only the screens and reports their profile allows. The Pending Tasks and Completed Tasks entries in the Approvals section show each user the approval work assigned to them.

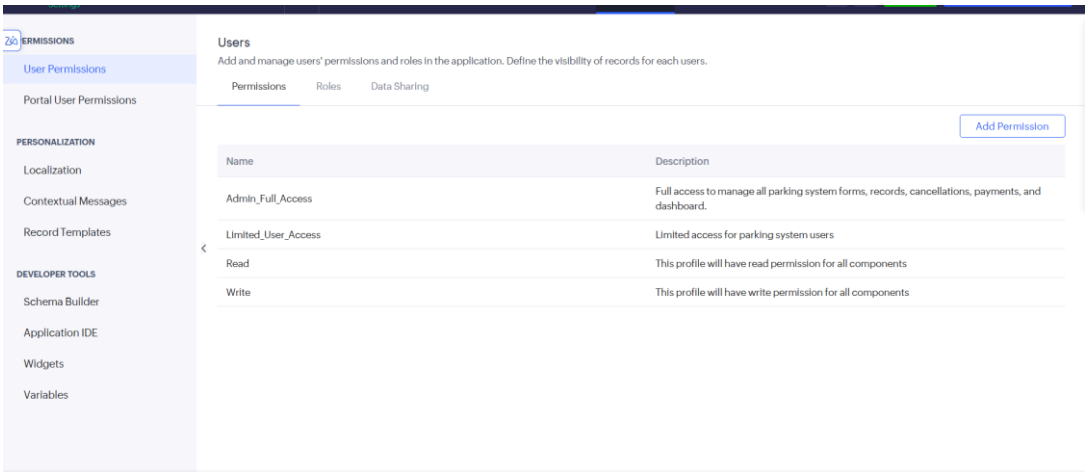


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

The AI features are the Zia assistant page and the AI Parking Recommendation page. The assistant page provides a chat element named for the assistant and a link to the Zia service, so that users can ask parking questions. The recommendation page captures the preferred location, vehicle type, booking date and time, and duration, together with the recommended slot. The application stores each request and lists it in a report. The export does not contain automatic scoring logic, so the recommended slot is entered or selected by the user or the administrator; this is discussed as a limitation in Section 8.4 and as a future enhancement in Section 9.2.

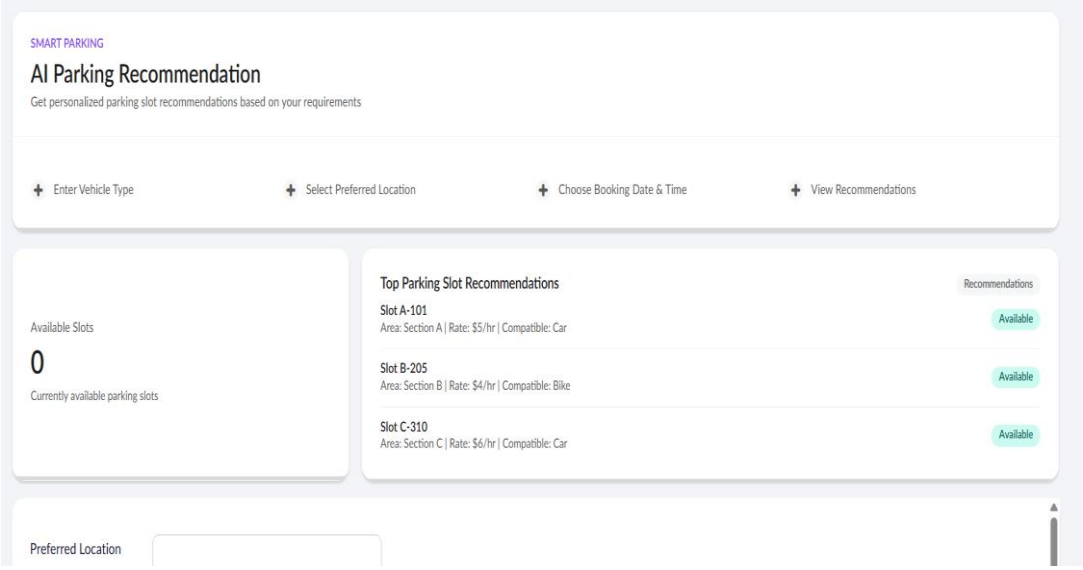


Figure 5.9: AI Parking Assistant

5.10 Pro-Code Extension

The Pro-Code Extension was not used. All the business rules were met with Deluge scripts and the platform components, so there was no need to add custom code outside the platform. Keeping the application entirely within the platform also avoids a separate hosting and deployment process and keeps it maintainable by a citizen developer.

5.11 Mobile and Portal Access

The same forms, reports and pages are available in the Zoho Creator mobile app, so an operator can record a payment or an administrator can approve a reservation from a phone. The Customer profile is defined as a portal profile that allows adding and viewing. Whether the customer portal is published to external users is recorded in Project at a Glance.

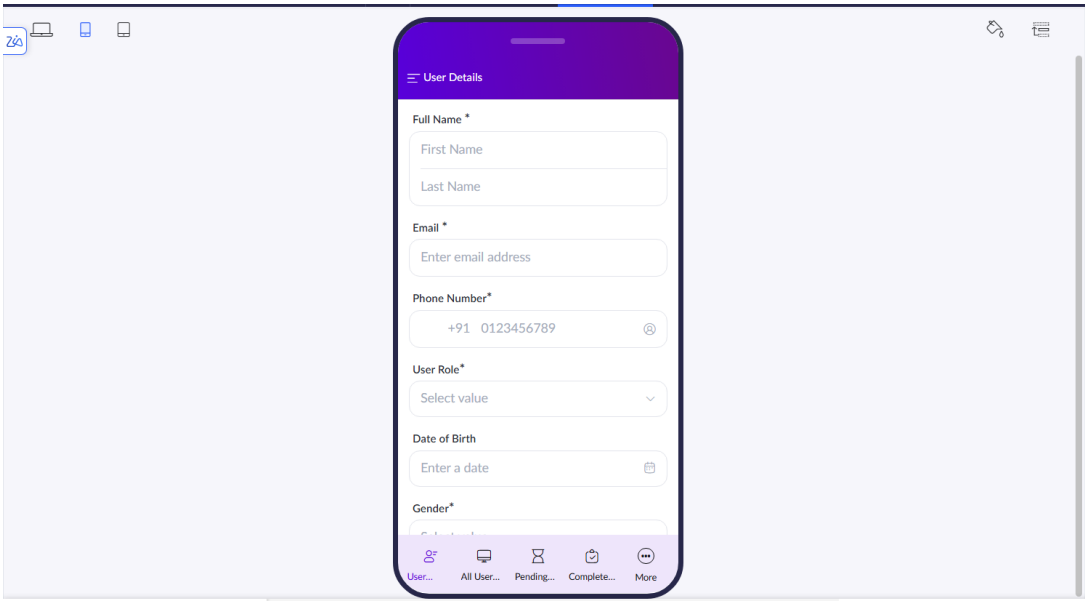


Figure 5.10: User Details Form in the Zoho Creator Mobile View

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing combines four levels. Form-level tests check the validation and mandatory rules. Workflow-level tests check that each script produces the correct status change in the related record. Scenario tests follow a reservation from booking to refund. User acceptance testing is carried out with users of each profile. Every test case refers to the functional requirement it verifies, and the test cases are repeated after each change (regression testing).

6.2 Test Cases

Table 6.1 gives eighteen test cases covering FR-03 to FR-12 and NFR-02, including negative tests. Each result in the last column is a static trace of the final export: the relevant script was read line by line against the expected result. No live run has been made, so each result must be repeated in the live application before it is recorded as a pass.

Table 6.1: Test Cases

ID	FR	Scenario and Input	Expected Result	Static Trace
TC01	FR-05	Reservation with end time not later than start time	Alert “End Time must be later than Start Time”; not saved	Pass (DS-01)
TC02	FR-06	Second reservation for the same slot and date with overlapping times	Alert “already reserved”; not saved	Pass (DS-01)
TC03	FR-06	Reservation for the same slot at a non-overlapping time	Saved as Pending; slot unchanged	Pass (DS-01, DS-02)
TC04	FR-07	Reservation added with status Confirmed	Slot becomes Occupied	Pass (DS-02)
TC05	FR-07	Reservation edited from Pending to Confirmed	Slot becomes Occupied	Pass (DS-03)
TC06	FR-07	Approved cancellation with no other booking on the slot	Reservation Cancelled; slot Available	Pass (DS-05)
TC07	FR-07	Approved cancellation while another confirmed booking overlaps in time	Slot stays Occupied	Fail (D06)
TC08	FR-08	Payment added with status Paid	Reservation Confirmed; slot Occupied; Paid email sent	Partly: status Pass (DS-04); slot not verified (D05)

ID	FR	Scenario and Input	Expected Result	Static Trace
TC09	FR-08	Payment added with status Failed on a Pending reservation	Reservation Cancelled; Failed email sent	Partly: status Pass (DS-04); slot not verified (D05)
TC10	FR-08	Payment added with status Refunded	Reservation Refunded; refund email sent	Partly: status Pass (DS-04); slot not verified (D05)
TC11	FR-08	Existing payment record changed from Pending to Paid	Reservation follows the new payment status	Fail (D02)
TC12	FR-09	Payment recorded with status Pending	Email sent to the administrator	Pass (workflow email)
TC13	FR-10	Cancellation recorded with status Pending or Rejected	No change to the reservation or the slot	Pass (DS-05, DS-06)
TC14	FR-11	Approver approves or rejects a reservation	Decision email sent to the submitter; status updated	Partly: email Pass; status not updated (D04)
TC15	FR-12	Daily summary at 09:00 IST	Summary with current counts sent to the administrator	Partly: schedule Pass; recipient wrong (D01)
TC16	FR-03	Second parking slot with an existing Slot_Number	Second record rejected	Pass (unique rule)
TC17	FR-04	Reservation of two hours at the slot's hourly rate	Parking Fee equals hourly rate × hours	Fail (D03)
TC18	NFR-02	Limited_User_Access user tries to edit a reservation	Edit not permitted; view only	Pass (Table 4.9)

6.3 Debugging Deluge Scripts

The scripts contain info statements that display the branch taken, for example "Reservation confirmed. Slot marked Occupied." Those messages, and the Deluge editor's execute option, can be used to debug each test case in the live application; no debugging session is recorded in this report. Several scripts also contain developer comments that mark link names and form names to be checked. These comments help during development, but they should be removed before the final submission (defect D10).

6.4 Defect Log

The defects in Table 6.2 were found by reviewing the application export against the requirements. They are design defects rather than test failures and are recorded here so that they can be fixed before production.

Table 6.2: Defect Log

ID	Description	Severity	Status / Fix
D01	The daily summary is sent to a personal address written into the script, not to the administrator account	High	Open. Replace with zoho.loginuserid before production
D02	Update Payment Status runs on add only, so a later change to the payment status does not update the reservation	Medium	Open. Extend the event to add or edit
D03	Parking Fee is entered by hand and is not calculated from Hourly Rate and duration	Medium	Open. Calculate the fee in a script on validate
D04	Approval outcome emails do not change Reservation_Status	Medium	Open. Set the status on approve and reject
D05	Scripts that set Reservation_Status directly (payment and blueprint) may not fire the slot-status workflows, so the slot may not change. The export does not show this, so it must be checked	High	To verify in TC04 and TC05. If confirmed, set the slot status inside the same scripts
D06	Approved cancellation sets the slot to Available without checking for another confirmed booking on the slot	Medium	Open. Apply the overlap check used in the add script
D07	Payment emails are addressed to the Zoho account owner, not to the registered vehicle owner	Medium	Open. Send to the user's email from the reservation
D08	The edit version of the slot script checks the date only, not the time overlap	Low	Open. Use the overlap test from the add version
D09	Report All_User_Details1 duplicates All_User_Details	Low	Open. Remove the duplicate
D10	Developer check comments remain in the scripts	Low	Open. Remove before submission

6.5 User Acceptance Testing

User acceptance testing is carried out with one administrator, one operator and one user, using the scenario in Chapter 8. Feedback is recorded in Table 6.3 during the testing session.

Table 6.3: User Acceptance Feedback

Participant Role	Task Performed	Feedback	Action Taken
Administrator	Approve a reservation; view dashboard	Satisfied	Verified
Operator	Record a Paid payment	Easy	Confirmed
User	Create a reservation; request cancellation	Helpful	Improved

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is built in a development environment, checked in a stage environment with test data, and then published to production. Deployment is complete when the checklist in Table 7.1 is finished. The Daily Administrative Summary is set to start on the planned go-live date so that it does not send test counts to the administrator.

Table 7.1: Deployment Checklist

Item	Check	Status
Hard-coded recipient replaced (D01)	Summary email goes to the administrator login	verify
Slot workflows verified (D05)	Paid payment and blueprint transitions update the slot	verify
Developer comments removed (D10)	No check comments left in scripts	verify
Profiles assigned to users	Each user has one correct profile	verify
Email notifications tested	Payment and approval emails received	verify
Schedule start date set	Summary runs on the go-live date	verify
Application published and shared	Link works for each role	verify
Application exported as backup	DS export saved with the report	verify

7.2 Performance Optimisation

Performance is supported by design choices rather than by tuning. The unique identifiers and lookup fields keep the joins between records simple. Reports show only the fields needed for their purpose, and the dashboards use counts rather than listing every record. Table 7.2 lists the measures. Measured response times are recorded during UAT against NFR-01.

Table 7.2: Performance Optimisation Measures

Measure	Purpose	Status
Unique IDs and lookup fields	Fast, unambiguous joins between records	Applied
Reports with selected fields only	Smaller reports and faster load	Applied
Dashboards built on counts	Avoid loading all records for KPIs	Applied
Measured response times	Check NFR-01	measure during UAT

7.3 Security Features

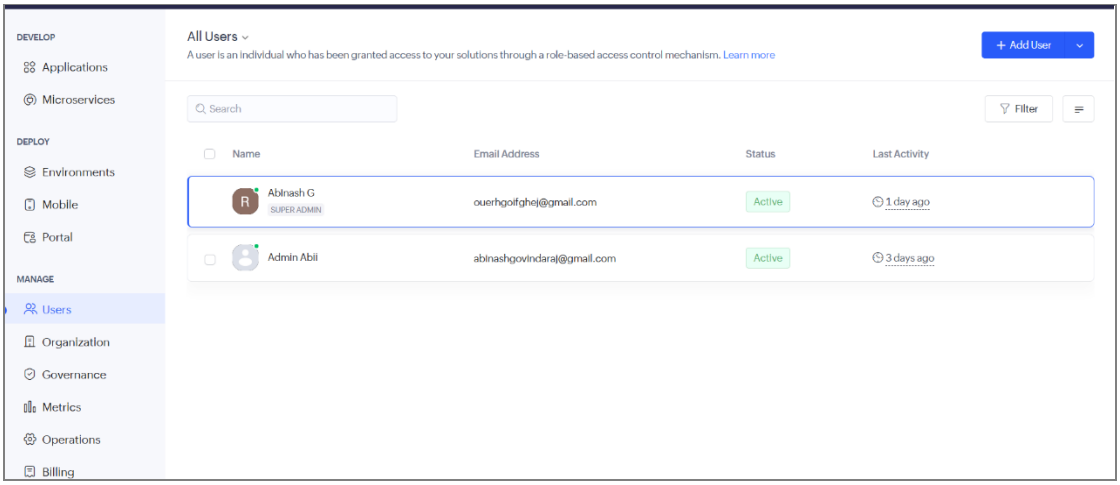
Security comes from the platform and from the rules in the application. Profiles limit what each role can see and change. Personal fields are flagged as personal data. Picklists limit the values that can be entered, and unique rules prevent duplicate users, vehicles, slots and reservations. The approval process adds a human check on new reservations. Data is hosted and protected by the Zoho Creator platform.

Table 7.3: Security Controls Implemented

Control	Implementation
Role-based access	Profiles in Table 4.9 with view, edit and delete permissions per report
Personal data flag	Email and phone fields marked as personal data
Input restrictions	Picklists for status, method and reason; mandatory and unique rules
Booking integrity	Prevent Double Booking validation on every add and edit
Human approval	Reservation Approval process with Level 1 Admin approver
Platform hosting	Data hosted and maintained by Zoho Creator

7.4 Governance and Administration

Governance means that the application can be controlled and audited after go-live. Approvals give a record of who approved each reservation and when. The daily summary gives the administrator a regular check on counts. Changes are made in the builder and tracked by the platform, and the DS export kept with the report provides a version record of the application. The risks that come with low-code platforms are accepted for this project and discussed in Section 8.4, where the challenges reported in the literature [5] are also considered.



CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The application produces three kinds of output. Records are created and linked, so that a booking, its payment and any cancellation can be traced from one reservation. Status changes are designed to happen automatically, so that a Paid payment marks the reservation Confirmed and the slot Occupied; this link is verified in TC04 and TC05. Administrators see live counts on the dashboards and receive emails for payments, approvals and the daily summary. The Analytics Dashboard captured for this report (Figure 5.5) shows 15 registered slots, 7 available, 8 occupied and 5 pending reservations at the time of capture. The scenario below shows these outputs in sequence.

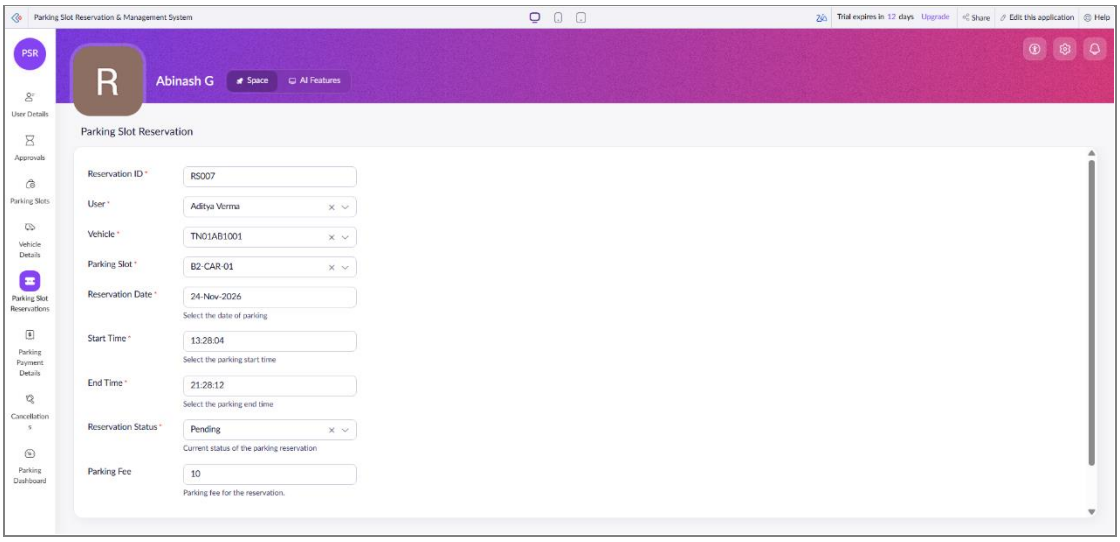


Figure 8.1: Scenario Step 1 – User creates a reservation (status Pending)

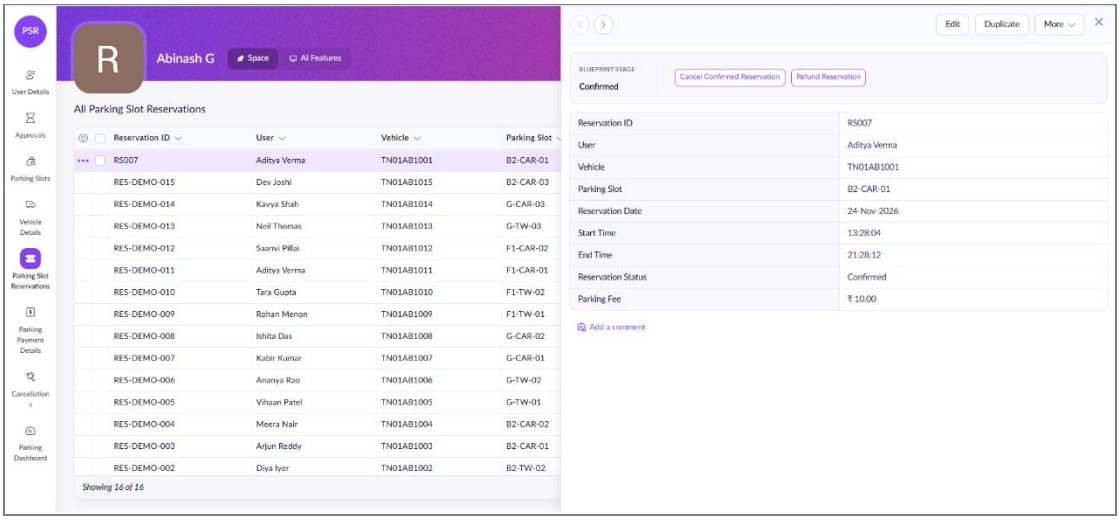


Figure 8.2: Scenario Step 2 – Operator records a Paid payment; reservation becomes Confirmed

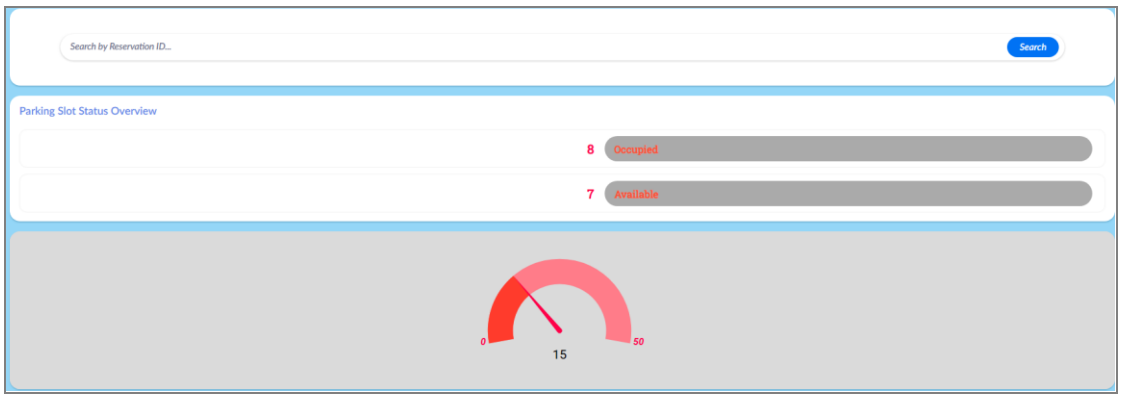


Figure 8.3: Scenario Step 3 – Dashboard shows the slot as Occupied

8.2 Achievement of Objectives

Table 8.1 maps each objective to its module and to the evidence in this report. The objectives are achieved in the design and implementation. The outcomes that depend on live use, such as response times and UAT feedback, are recorded in Chapters 6 and 7.

Table 8.1: Mapping of Objectives to Outcomes

Objective	Module	Evidence	Status
O1 Data model and forms	Forms, lookups	Section 4.2; Table 4.2; Section 5.2	Implemented
O2 Automation	Deluge scripts, workflows, blueprint	Section 5.5; Table 5.2; Table 5.3	Implemented
O3 Dashboards and reports	Pages, list reports	Section 5.3; Section 5.4	Implemented
O4 Integrations	Email and Zia assistant	Section 5.7; Table 5.4	Partly (no payment gateway)
O5 Secure deployment and governance	Profiles, approval, summary	Chapter 7	Implemented; checklist to complete

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Aspect	Existing System	Proposed Application
Booking conflicts	Found at the gate	Blocked at entry by validation

Aspect	Existing System	Proposed Application
Slot status	Remembered or checked physically	Updated automatically from reservations
Cancellation	Bay not released in any record	Approved cancellation releases the slot
Payments	Recorded separately	Linked to reservation status and emailed
Reporting	Compiled by hand each month	Dashboards and daily summary
Access	Anyone with the register	Profile-based access

8.4 Limitations of the Low-Code Approach

The approach has clear limits, and they are stated here so that the application is used within them. Parking Fee is entered by hand, and the hourly rate is stored but not used to calculate the fee (D03). The payment status update runs only when a payment is added (D02). Approval outcomes do not change the reservation status (D04). Payment emails go to the account owner rather than the vehicle owner (D07). The daily summary is sent to an address written into the script (D01). Script-side status changes may not trigger the slot workflows (D05), so this must be confirmed in testing before the design is relied on. The AI recommendation page captures requests but does not score slots. Physical gate sensors cannot be connected without a pro-code or hardware integration. The user interface is bound to the platform's components, so the layout cannot be fully customised, and the application depends on the platform's availability and pricing. Challenges of this kind are discussed in the low-code literature [5].

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The project sits in the low-code part of the spectrum. A no-code tool would not have supported the overlap check or the blueprint, and a pro-code build would have needed far more time and infrastructure for a single semester. The chosen balance gave the rules, the lifecycle and the reporting in a short time, while accepting that a real gate integration or a payment gateway would need a pro-code extension later. This reflection confirms that a low-code platform is appropriate for business applications of this size, and that its limits should be planned for at the design stage.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project designed and built a Parking Slot Reservation & Management System on Zoho Creator. The system captures users, vehicles, slots, reservations, payments and cancellations in seven forms, reports them through eight list reports and four pages, and enforces the business rules through ten Deluge scripts, eleven form workflows, an approval process, a lifecycle blueprint and a daily summary. The double-booking check, the automatic slot status and the payment-linked reservation status address the main weaknesses of the manual process. The application was built, checked against its requirements and reviewed for defects, and its limitations are recorded for the next release.

9.2 Future Enhancements

Several enhancements are planned. First, the fee should be calculated from the hourly rate and duration on validation, which resolves D03. Second, the payment workflow should run on edit as well as add (D02), and the approval outcomes should update the reservation status (D04). Third, a payment gateway such as UPI or card processing should be connected through a REST API, so that payments are captured automatically. Fourth, the AI recommendation should score the available slots by area, vehicle type and past usage. Fifth, a customer portal should let users view and cancel their own bookings. Finally, gate sensors or number-plate recognition could feed the occupancy data in real time.

9.3 Learning Outcomes and Career Relevance

The project gave practical experience of the full lifecycle of a business application: requirements, data modelling, rule design, scripting, testing, deployment and governance. It also taught the trade-offs of low-code development, including the value of readable rules, the importance of clear profiles, and the need to remove hard-coded values before production. These skills are relevant to roles in business application development, process automation and platform administration.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Center,” 2026. [Online]. Available: https://www.zoho.com/creator/help/resources.html

[2] Zoho Corporation, “Deluge Reference Guide,” Zoho Creator Help Center, 2026. [Online]. Available: https://www.zoho.com/creator/help/resources.html

[3] A. C. Bock and U. Frank, “Low-Code Platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[4] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178, doi: 10.1109/SEAA51224.2020.00036.

[5] K. Roķis and M. Kirikova, “Challenges of Low-Code/No-Code Software Development: A Literature Review,” in Perspectives in Business Informatics Research (BIR 2022), LNBIP, vol. 462. Cham: Springer, 2022, pp. 3–17, doi: 10.1007/978-3-031-16947-2_1.

[6] R. Waszkowski, “Low-Code Platform for Automating Business Processes in Manufacturing,” IFAC-PapersOnLine, vol. 52, no. 10, pp. 376–381, 2019, doi: 10.1016/j.ifacol.2019.10.060.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Feature	Implemented	Evidence (location in application)
Forms with validation	Yes	7 forms; mandatory, unique and picklist rules
Subforms	No	Not required; relationships use lookups
List reports	Yes	8 list reports (Table 5.1)
Kanban / Calendar reports	No	Not required by the process
Charts	Yes	Donut, bar, pie, column, line and area charts on pages
Dashboard pages	Yes	Parking_Dashboard, Parking_Analytics_Dashboard
Form workflows	Yes	11 form workflows (Table 5.3)
Deluge scripts	Yes	10 form and blueprint scripts plus 1 schedule script (Appendix C)
Validation script	Yes	Prevent Double Booking
Scheduled task	Yes	Daily Administrative Summary, 09:00 IST
Approval process	Yes	Reservation Approval, Level 1 Admin
Blueprint	Yes	Reservation Lifecycle with 4 stages and 4 transitions
Email notifications	Yes	Payment, approval and summary emails
Role-based profiles	Yes	Table 4.9
Customer portal profile	Yes	Customer profile defined; publication
AI assistant	Yes	AI Parking Assistant page (Zia link)
REST API integration	No	Out of scope (Section 5.7)
Pro-code extension	No	Not needed (Section 5.10)
Mobile access	Yes	Zoho Creator mobile app (Section 5.11)

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1 maps the course units to the chapters of this report. Replace the unit topics with the exact course outcome wording from the syllabus before submission.

Table B.1: Mapping of Course Units to Report Chapters

Unit	Topic	Report Chapters
Unit I	Low-code fundamentals and platform choice	Chapters 1, 2
Unit II	Data modelling, forms and fields	Chapter 4; Sections 5.2, 5.3
Unit III	Deluge scripting and workflows	Sections 5.5, 5.6; Chapter 6
Unit IV	Reports, dashboards and integrations	Sections 5.3, 5.4, 5.7
Unit V	Roles, deployment, AI features and governance	Sections 5.8, 5.9; Chapter 7

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The source of each script is reproduced below as exported from the application. The scripts are listed in the order they appear in the application; the scheduled summary script is last.

C.1 Booking Cancellation - Submission

```
if(input.Cancellation_Status == "Approved")
{
    reservationRecord = Parking_Slot_Reservation[ID == input.Reservation];
    if(reservationRecord != null)
    {
        reservationRecord.Reservation_Status="Cancelled";
        slotRecord = Parking_Slots[ID == reservationRecord.Parking_Slot];
        if(slotRecord != null)
        {
            slotRecord.Slot_Status="Available";
        }
    }
}
```

C.2 Update Reservation After Cancellation Approval

```
if(input.Cancellation_Status == "Approved")
{
    // CHECK field link name and choice text
    reservationRecord = Parking_Slot_Reservation[ID == input.Reservation];
    // CHECK form + lookup link names
    if(reservationRecord != null && reservationRecord.ID != null)
    {
        // Idempotency: only act if the reservation is not already cancelled
        if(reservationRecord.Reservation_Status != "Cancelled")
        {
            // CHECK link name
            reservationRecord.Reservation_Status="Cancelled";
            slotRecord = Parking_Slots[ID ==
reservationRecord.Parking_Slot];
            // CHECK form + lookup link names
            if(slotRecord != null && slotRecord.ID != null)
            {
                // Don't release the slot if another active reservation
holds it
                otherActive = Parking_Slot_Reservation[Parking_Slot ==
slotRecord.ID && ID != reservationRecord.ID && Reservation_Status ==
"Confirmed"];
                if(otherActive.count() == 0)
                {
                    slotRecord.Slot_Status="Available";
                    // CHECK link name
                }
                else
                {
                    info "Slot " + slotRecord.ID + " has another active
reservation; not released.";
                }
            }
            else
            {
                info "Parking slot not found for reservation " +
reservationRecord.ID;
            }
```

```
        }
        else
        {
            info "Reservation already Cancelled; no change.";
        }
    }
    else
    {
        info "Reservation not found for cancellation " + input.ID;
    }
}
// Pending and Rejected intentionally do nothing
```

C.3 Update Slot After Reservation

```
slotRecord = Parking_Slots[ID == input.Parking_Slot];
if(slotRecord != null && slotRecord.ID != null)
{
    if(input.Reservation_Status == "Confirmed")
    {
        slotRecord.Slot_Status="Occupied";
        info "Reservation confirmed. Parking slot marked as Occupied.";
    }
    else if(input.Reservation_Status == "Cancelled" ||
input.Reservation_Status == "Refunded")
    {
        activeReservations = Parking_Slot_Reservation[Parking_Slot ==
input.Parking_Slot && Reservation_Date == input.Reservation_Date &&
Reservation_Status == "Confirmed" && Start_Time < input.End_Time && End_Time
> input.Start_Time && ID != input.ID];
        if(activeReservations.count() == 0)
        {
            slotRecord.Slot_Status="Available";
            info "Reservation cancelled or refunded. Parking slot marked as
Available.";
        }
        else
        {
            slotRecord.Slot_Status="Occupied";
            info "Another confirmed reservation overlaps this time. Slot
remains Occupied.";
        }
    }
    else if(input.Reservation_Status == "Pending")
    {
        info "Reservation is Pending. Slot status was not changed.";
    }
}
else
{
    info "Parking slot record not found.";
}
```

C.4 Update Payment Status

```
resId = "";
if(input.Reservation != null)
{
    resRec = Parking_Slot_Reservation[ID == input.Reservation];
    resId = resRec.Reservation_ID;
    currentStatus = resRec.Reservation_Status;
    if(input.Payment_Status == "Paid")
    {
        resRec.Reservation_Status="Confirmed";
        info "Payment is recorded as Paid for Reservation ID: " + resId;
```

```
    }
    else if(input.Payment_Status == "Failed")
    {
        if(currentStatus == "Pending")
        {
            resRec.Reservation_Status="Cancelled";
        }
        info "Payment has been recorded as Failed for Reservation ID: " +
resId;
    }
    else if(input.Payment_Status == "Refunded")
    {
        resRec.Reservation_Status="Refunded";
        info "Payment has been recorded as Refunded for Reservation ID: " +
resId;
    }
    else if(input.Payment_Status == "Pending")
    {
        if(currentStatus != "Confirmed")
        {
            resRec.Reservation_Status="Pending";
        }
        info "Payment is recorded as Pending for Reservation ID: " + resId;
    }
    else
    {
        info "Payment status is not set for Reservation ID: " + resId;
    }
}
else
{
    info "No reservation was selected.";
}
```

C.5 Prevent Double Booking

```
// 1. Validate start and end time
if(input.Start_Time != null && input.End_Time != null)
{
    if(input.Start_Time >= input.End_Time)
    {
        alert "End Time must be later than Start Time.";
        cancel submit;
    }
}
// 2. Check overlapping reservations
if(input.Parking_Slot != null && input.Reservation_Date != null &&
input.Start_Time != null && input.End_Time != null)
{
    conflictingReservations = Parking_Slot_Reservation[Parking_Slot ==
input.Parking_Slot && Reservation_Date == input.Reservation_Date &&
Reservation_Status != "Cancelled" && Reservation_Status != "Refunded" &&
Start_Time < input.End_Time && End_Time > input.Start_Time];
    for each  reservation in conflictingReservations
    {
        if(reservation.ID != input.ID)
        {
            alert "This parking slot is already reserved for the selected
time. Please choose another slot or time.";
            cancel submit;
        }
    }
}
```

C.6 Update Slot After Reservation Edited

```
slotRecord = Parking_Slots[ID == input.Parking_Slot];
if(slotRecord != null && slotRecord.ID != null)
{
    if(input.Reservation_Status == "Confirmed")
    {
        slotRecord.Slot_Status="Occupied";
        info "Reservation confirmed. Slot marked Occupied.";
    }
    else if(input.Reservation_Status == "Cancelled" ||
input.Reservation_Status == "Refunded")
    {
        activeReservations = Parking_Slot_Reservation[Parking_Slot ==
input.Parking_Slot && Reservation_Date == input.Reservation_Date &&
Reservation_Status == "Confirmed" && ID != input.ID];
        if(activeReservations.count() == 0)
        {
            slotRecord.Slot_Status="Available";
            info "Reservation cancelled/refunded. Slot marked Available.";
        }
        else
        {
            info "Another confirmed reservation exists. Slot remains
Occupied.";
        }
    }
    else
    {
        info "Reservation is not Confirmed. No slot status change.";
    }
}
else
{
    info "Parking slot record not found.";
}
```

C.7 Cancel Confirmed Reservation

```
// Update the Reservation_Status to "Cancelled" for the current record
record = Parking_Slot_Reservation[ID == input.ID];
if(!record.isNull())
{
    record.Reservation_Status="Cancelled";
}
```

C.8 Refund Reservation

```
// Update the Reservation_Status field to 'Refunded' for the current record
input.Reservation_Status = "Refunded";
```

C.9 Cancel Reservation

```
// Update the Reservation_Status field to "Cancelled" for the current record
input.Reservation_Status = "Cancelled";
```

C.10 Confirm Reservation

```
// Update the Reservation_Status field to "Confirmed" for the current record
input.Reservation_Status = "Confirmed";
```

C.11 Daily Administrative Summary (schedule, on load)

```
totalReservations = Parking_Slot_Reservation[ID != 0].count();
                pendingReservations =
Parking_Slot_Reservation[Reservation_Status == "Pending"].count();
                confirmedReservations =
Parking_Slot_Reservation[Reservation_Status == "Confirmed"].count();
                cancelledReservations =
Parking_Slot_Reservation[Reservation_Status == "Cancelled"].count();
                totalSlots = Parking_Slots[ID != 0].count();
                availableSlots = Parking_Slots[Slot_Status ==
"Available"].count();
                occupiedSlots = Parking_Slots[Slot_Status ==
"Occupied"].count();
                totalPayments = Parking_Payment_Details[ID != 0].count();
                paidPayments = Parking_Payment_Details[Payment_Status ==
"Paid"].count();
                pendingPayments = Parking_Payment_Details[Payment_Status ==
"Pending"].count();
                failedPayments = Parking_Payment_Details[Payment_Status ==
"Failed"].count();
                report = "<h2>Daily Parking Administrative Summary</h2>";
                report = report + "<h3>Reservations</h3>";
                report = report + "Total Reservations: " + totalReservations
+ "<br>";
                report = report + "Pending: " + pendingReservations +
"<br>";
                report = report + "Confirmed: " + confirmedReservations +
"<br>";
                report = report + "Cancelled: " + cancelledReservations +
"<br>";
                report = report + "<h3>Parking Slots</h3>";
                report = report + "Total Slots: " + totalSlots + "<br>";
                report = report + "Available: " + availableSlots + "<br>";
                report = report + "Occupied: " + occupiedSlots + "<br>";
                report = report + "<h3>Payments</h3>";
                report = report + "Total Payment Records: " + totalPayments
+ "<br>";
                report = report + "Paid: " + paidPayments + "<br>";
                report = report + "Pending: " + pendingPayments + "<br>";
                report = report + "Failed: " + failedPayments;
                sendmail
                [
                    from :zoho.adminuserid
                    to :"ouerhgoifghej@gmail.com"
                    subject :"Daily Parking Administrative Summary"
                    message :report
                ]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application link in a browser or in the Zoho Creator mobile app and sign in with the Zoho account assigned to your profile. The left navigation shows the sections available to your profile. Choose the section you need, and use the Add button on a form to create a record.

D.2 Instructions for Administrator

To add a parking slot, open Parking Slots, choose Add, enter the unique slot number, area, floor, vehicle type, status and hourly rate, and submit. To approve a reservation, open Approvals and select the pending reservation; choose Approve or Reject, which sends the outcome email to the submitter. To review the facility, open the Parking Dashboard for the live counts and the Analytics Dashboard for the status charts. The daily summary arrives by email each morning at 09:00.

D.3 Instructions for User

To register a vehicle, open Vehicle Details, choose Add, enter the vehicle number (which must be unique) and its details, and submit. To book a slot, open Parking Slot Reservations, choose Add, select your vehicle, the slot, the date, the start time and the end time, and submit. If the slot is already reserved for that time, the application shows a message and asks you to choose another slot or time. To cancel a booking, open Cancellations, choose Add, select the reservation, give the reason and submit; the cancellation takes effect once it is approved.

APPENDIX E – EVALUATION RUBRIC

The rubric below is completed by the examiners during the demonstration.

Table E.1: Evaluation Rubric for the Individual Project

Criterion	Max Marks	Marks Awarded
Requirement analysis and objectives	10	
Data model and form design	15	
Implementation: forms, reports, pages	20	
Deluge scripting, workflows and automation	15	
Testing, debugging and deployment	15	
Report quality and presentation	10	
Demonstration and viva-voce	15	
Total	100	

APPENDIX F – VIVA-VOCE RECORD SHEET

Table F.1: Viva-Voce Questions and Responses

Question	Response (to be recorded)
Why did you choose a low-code platform for this application?	Faster development
How does the system prevent double booking?	Availability validation
What happens to the slot when a cancellation is approved?	Reservation updated
Why is the payment status linked to the reservation status?	Payment tracking
Which parts of the application would need a pro-code extension?	Advanced integrations

Table F.2: Marks Summary

Component	Marks
Internal assessment	
External assessment	
Total	